



KaliBench: A Fine-Grained Benchmark for Cybersecurity Tool Use on Kali Linux with Runtime-Free Verifiable Rewards

Pengfei Li^{1,*}, Naufal Suryanto^{1,*}, Sicheng Zhang¹, Muzammal Naseer^{1,2}

¹Khalifa University, ²University of Western Australia, *Equal contribution

LLMs are increasingly applied to cybersecurity workflows, where they are expected to translate analysts' intent into tool invocations. However, existing evaluations focus on knowledge-based assessments or end-to-end agentic tasks, and do not directly measure LLMs' ability to generate executable commands for real-world cybersecurity tools. This gap is critical because cybersecurity operations rely on strict command-line interfaces (CLIs), where minor syntax errors, incorrect flag-value bindings, or argument misordering can invalidate execution. We introduce KaliBench, a fine-grained benchmark and dataset for natural-language-to-CLI translation on Kali Linux, comprising 8,504 query-command pairs spanning 1,642 tools across 23 capability dimensions and 5 security phases. KaliBench is constructed via a manuscript-grounded pipeline with deterministic canonicalization and alias-aware evaluation, enabling precise and reproducible assessment of tool selection and argument construction. To ensure both semantic correctness and practical executability, we develop a multi-stage verification pipeline that combines LLM-based validation, sandboxed terminal execution, and human-in-the-loop refinement. Building on these fine-grained, deterministic signals, KaliBench further enables runtime-free verifiable rewards for training. Across three evaluation modes and 24 configurations of general-purpose and security-focused open-weight models, no open-weight model exceeds 42% exact-command accuracy in the unrestricted setting, highlighting the difficulty of accurate CLI-based cybersecurity tool use without explicit tool hints. We further show that supervised fine-tuning and reinforcement learning with verifiable rewards derived from KaliBench significantly improve an 8B model and achieve performance comparable to a 685B MoE model.

Project Page: <https://risys-lab.github.io/KaliBench/>
Github: <https://github.com/RISys-Lab/KaliBench>

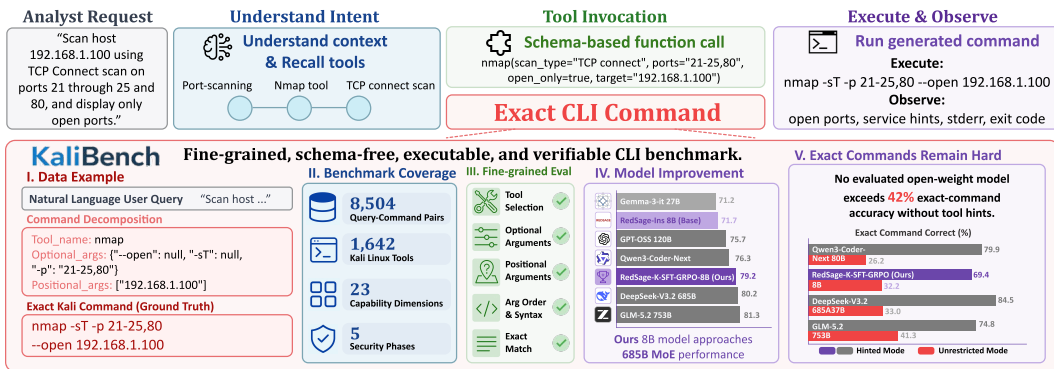


Figure 1: **Why KaliBench Matters?** KaliBench enables fine-grained, verifiable evaluation and runtime-free reward training for translating analyst intent into executable Kali commands.

1 Introduction

Large language models (LLMs) are increasingly adopted in cybersecurity workflows, ranging from interactive assistants to autonomous agents [40]. Early benchmarks in this domain primarily focus on *knowledge-based* assessment, testing models’ understanding of cybersecurity concepts via multiple-choice [36, 19] or structured-answer questions [14, 3]. More recent benchmarks move toward *end-to-end agentic* evaluation, measuring model performance in complex environments such as autonomously solving CTF-style challenges [32, 45]. While these benchmarks provide valuable insight into what LLMs know and how they behave in integrated workflows, they do not directly evaluate whether models can generate correct invocations of concrete cybersecurity tools in isolation.

This limitation is consequential because modern security operations rely heavily on large, heterogeneous command-line toolchains. Kali Linux, a widely used platform among cybersecurity practitioners, provides access to over 2,000 pre-installed security tool packages [31]. In practice, analysts must translate high-level goals (e.g., “scan this subnet for SMB misconfigurations”) into precise command-line invocations of tools such as *Nmap* for scanning, *Metasploit* for exploitation, and *Volatility* for memory forensics. Although LLMs with function-calling capabilities aim to automate this process, CLIs are unforgiving: minor errors in argument order, flag spelling, alias misuse, or tool misinterpretation can invalidate execution. This motivates a central question: *Can current LLMs reliably translate natural-language security requests into exact, executable CLI commands?*

While function-calling benchmarks for LLMs have matured rapidly [17, 10, 21, 20, 28], evaluating cybersecurity tool invocation poses distinct challenges. Existing benchmarks primarily assume schema-defined tools, in which functionality and parameters are explicitly specified via JSON schemas or APIs. In contrast, cybersecurity tooling is dominated by CLIs, such as the Kali Linux toolset, where enumerating hundreds of tool schemas in a model prompt is impractical in typical deployment settings due to context length limitations. As a result, Natural-Language-to-CLI (NL-to-CLI) evaluation is effectively *schema-free*: models must infer appropriate tools and arguments and generate syntactically correct, executable commands rather than populate structured fields.

To address this gap, we develop **KaliBench**, a fine-grained benchmark for NL-to-CLI cybersecurity tool use on Kali Linux. As shown in Fig. 1, KaliBench maps analyst intent to exact CLI commands, decomposing tool selection and argument construction for fine-grained, verifiable evaluation. It organizes tools across capability dimensions and security phases and is constructed via a manuscript-grounded pipeline. To ensure data quality, we employ a multi-stage verification process combining LLM-based validation, sandboxed execution, and human-in-the-loop refinement, producing semantically correct and executable commands.

KaliBench provides fine-grained supervision over tool selection and argument construction. We evaluate models under three settings, *unrestricted*, *restricted*, and *hinted*, which vary the degree of tool information provided. This structured formulation further enables *runtime-free verifiable rewards*, allowing training without command execution. Using this framework, we evaluate 24 configurations of general-purpose and security-focused open-weight models and find that performance remains limited, with argument construction as the primary bottleneck. We further show that Supervised Fine-Tuning (SFT) and Reinforcement Learning with Verifiable Rewards (RLVR) improve an 8B model and achieve performance competitive with substantially larger models. Our main contributions are summarized as follows:

- **A fine-grained benchmark for schema-free NL-to-CLI evaluation.** KaliBench is, to the best of our knowledge, the first benchmark for evaluating LLMs on cybersecurity tool invocation in schema-free CLI settings, comprising 8,504 query–command pairs over 23 capability dimensions and five security phases, split into 5,000 evaluation and 3,504 training samples after deduplication.
- **A verified and executable dataset.** We construct a high-quality dataset through a multi-stage verification pipeline that ensures both semantic correctness and practical executability.
- **Runtime-free verifiable rewards.** We show that deterministic CLI structure enables verifiable reward signals without runtime execution, supporting scalable reinforcement learning.
- **Fine-grained evaluation and training gains.** We evaluate over 20 LLMs under three settings and identify argument construction as the primary bottleneck. Training with supervised fine-tuning and reinforcement learning using KaliBench rewards improves an 8B model’s mean Total Score across the three evaluation modes by 7.5 percentage points.

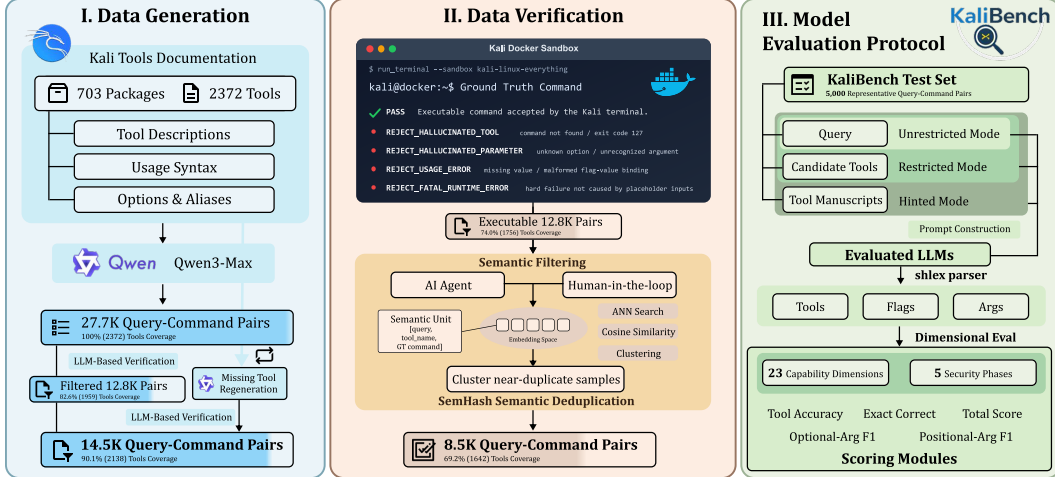


Figure 2: **KaliBench end-to-end pipeline for NL-to-CLI evaluation on Kali Linux.** The benchmark is constructed via LLM-based data generation, multi-round verification-regeneration, and representative-sample selection. Evaluation applies alias-aware parsing and fine-grained scoring over tools, arguments, and dimensional categories.

2 Related Work

LLMs and Cybersecurity Benchmarks. Early evaluations of LLMs focused on general-purpose knowledge as measured by MMLU [11]. HELM [18] later introduced a unified evaluation framework that aggregates benchmarks across broader dimensions. Although these evaluations characterize overall language understanding, they do not account for the domain-specific requirements or operational constraints encountered in cybersecurity settings.

As a result, several domain-specific benchmarks have been proposed to evaluate LLMs in cybersecurity contexts. Benchmarks such as SecEval [16], CyberMetric [36], CyberBench [19], SECURE [3], CS-Eval [44], SecBench [14], and CTI-Bench [1] primarily assess cybersecurity knowledge through multiple-choice, true/false, short-answer, or other classification-style questions spanning software and network security, cryptography, industrial control systems, and cyber-threat intelligence. While these benchmarks provide useful insight into models’ conceptual understanding, they largely evaluate knowledge recall and static reasoning rather than concrete operational behavior.

More recent work moves beyond static question answering toward *agentic* evaluation, where models interact with complex environments or autonomously solve multi-step tasks. For example, CyBench [45] and NYU-CTF Bench [32] evaluate LLMs in end-to-end security scenarios via Capture-the-Flag (CTF) challenges, while DefenderBench [46] studies language agents in enterprise-style cybersecurity environments with realistic defensive and offensive workflows. However, end-to-end task performance jointly depends on planning, environment interaction, observation interpretation, tool invocation, and error recovery, making it difficult to attribute failures to individual components. KaliBench complements these benchmarks by isolating command generation and separately evaluating tool selection, optional arguments, and positional arguments.

Function-Calling Benchmarks and Evaluations. Beyond agentic evaluation, a growing body of work studies LLMs’ ability to invoke external tools through explicit function-calling benchmarks. Benchmarks such as BFCL [28], ToolSandbox [21], StableToolBench [10], and HammerBench [38] evaluate tool use across single-turn, multi-turn, conversational, and agentic settings, emphasizing structured, schema-defined APIs and deterministic validation for general-purpose tool invocation. As a result, these benchmarks assume that tools and parameters are specified through explicit schemas, and do not consider settings where tools must be inferred from a shared CLI. To the best of our knowledge, no existing benchmark directly evaluates natural-language-to-CLI translation for real-world cybersecurity tools under such schema-free conditions, or provides fine-grained, alias-aware assessment of command correctness. **KaliBench** addresses this gap by enabling systematic evaluation of exact, executable CLI command generation for real-world cybersecurity toolchains.

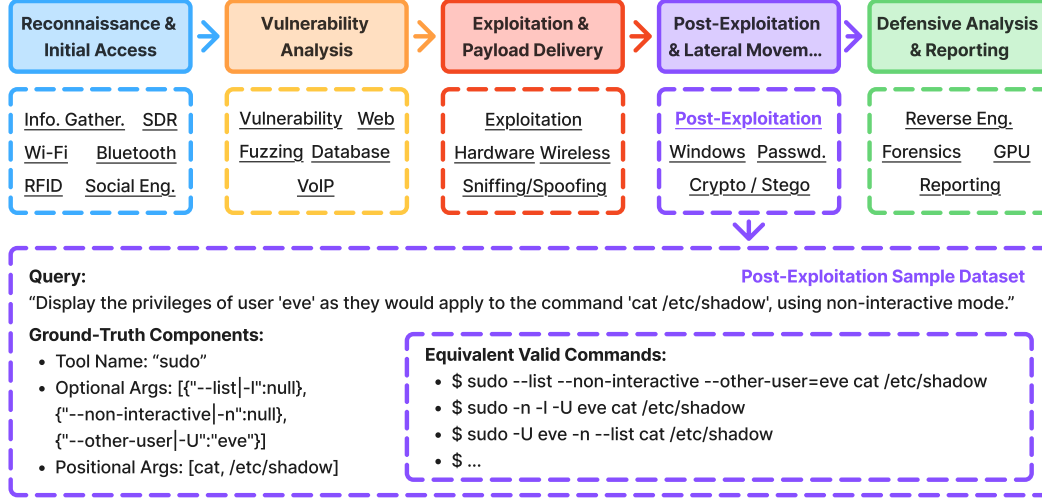


Figure 3: **KaliBench Structure and Representative Sample.** KaliBench is a structured benchmark of Kali Linux tools organized along the security attack lifecycle. It comprises 5 security phases and 23 functional dimensions. The dataset sample includes natural-language queries and decomposed ground-truth commands. See Appendix D for a detailed view of examples of all 23 dimensions.

3 KaliBench

KaliBench is a fine-grained benchmark and dataset that serves as a *translation layer* between natural-language cybersecurity queries and executable Kali Linux commands, spanning **1,642 tools** and **8,504** query–command pairs. Figure 2 illustrates the end-to-end pipeline for constructing, verifying, evaluating, and training with KaliBench. The pipeline proceeds in four stages: (1) we extract tool names, flag–value pairs, and documented aliases from official Kali Linux manuscripts, and use them as structured signals to guide LLM-based data generation; (2) generated candidates are iteratively filtered through LLM-based verification, sandboxed terminal execution, and human-in-the-loop evaluation; (3) the verified dataset is deduplicated and split into evaluation and training sets, enabling fine-grained evaluation across multiple metrics and capability dimensions; and (4) using the training split and verifiable reward signals, we demonstrate that parameter efficient post-training with supervised fine-tuning and reinforcement learning significantly improves model performance.

1. Data Generation. We curate official documentation for Kali Linux tools from the Kali Linux repository [15]. The source corpus comprises 2,372 tools described in structured Markdown, each specifying long and short options and relevant aliases. The documentation also provides a metapackage taxonomy, which we use to annotate each tool with its cybersecurity capability and functional dimension. We process the corpus at the tool level. For each tool, we use Qwen3-Max [30] to generate 10-13 natural-language query–command pairs, along with structured annotations including the tool name, optional arguments as alias-aware flag–value pairs, and positional arguments. These ground-truth command components enable fine-grained analysis of model performance in tool selection, argument construction, and syntactic validity under UNIX shell conventions. Representative examples are shown in Fig. 3, illustrating the diversity of KaliBench across security phases and functional dimensions. This stage produces 27.7K query–command pairs covering all documented tools. More details about the data generation process can be found in Appendix B.

2. Data Verification, Cleaning, and Deduplication. Despite grounding data generation in official Kali Linux tool manuscripts using a state-of-the-art model (Qwen3-Max), hallucination remains a major challenge. Our initial manual inspection shows that over 50% of generated commands contain hallucinated or misspelled options (e.g., `--silent` instead of `--silence`). To address this issue, we adopt a *multi-stage human-in-loop verification pipeline* that ultimately retains 8,504 high-quality query–command pairs after cleaning and deduplication.

LLM-as-Verifier. We prompt an LLM with the query, the generated_command, and the corresponding tool_manuscript, and task it with verifying (i) whether all flags and arguments are

documented, and (ii) whether the command faithfully implements the query using only supported functionality. After the first verification round, 53.8% of generated examples are filtered out. This leaves 413 tools (17.4%) without valid pairs; to preserve coverage, we iteratively regenerate and re-verify the data. The resulting LLM-validated dataset contains 14.5K query–command pairs, with 47.65% of the initial data removed. The prompt details are provided in Appendix C.2.

Terminal Verification. To further ensure the executability of ground-truth commands, we perform system-level validation by executing each LLM-verified command in a controlled sandbox. Specifically, we run each command inside a Docker container provisioned with a *Kali-linux-everything* image and record its outputs, including `stdout`, `stderr`, exit code, and execution timeouts. Because command parameters are synthetic (e.g., addresses, devices, paths, and files may not exist in the sandbox), we apply a rule-based filtering pipeline based on observed execution outputs. Commands are rejected when execution logs indicate errors caused by hallucinated tools or invalid arguments that were not detected by the LLM-as-Verifier. This process filters out 11.95% of the LLM-verified commands. Further details of the terminal verification process are provided in Appendix E.

Human-in-the-loop Evaluation. As a final verification step, we perform human-in-the-loop evaluation by analyzing model failures in the hinted setting (see the Evaluation Protocol). We focus on cases where even the strongest evaluated model fails to generate a correct command despite having access to tool documentation, as these cases are more likely to reflect issues in the dataset rather than model limitations. We use an AI agent¹ to produce initial verdicts by cross-checking the semantic alignment among the query, generated command, ground-truth command, and tool documentation. Human reviewers then examine these verdicts to identify query ambiguities and labeling errors. This process reveals that an additional 4.9% of the verified dataset is invalid due to: (a) Malformed CLI options (e.g., ‘--g’ instead of the documented ‘-g’), (b) Omission of arguments explicitly required by the query (e.g., generating ‘<tool> completion’ instead of the query-required ‘<tool> completion bash’), and (c) Semantic mismatches in argument use (e.g., choosing a flag whose effect does not match the requested operation, swapping the positions of input and output operands, or supplying a filesystem path where the command expects a URL). Detailed cases can be found in Appendix F.

Deduplication and Train–test Split. To construct a compact yet semantically diverse evaluation set, we perform *semantic selection* using the SemHash framework [37]. Each record is represented as a semantic unit [query, tool_name, ground_truth_command]. Records are embedded into a dense vector space, clustered via approximate nearest-neighbor search, and grouped when pairwise cosine similarity exceeds 0.90. From each cluster, we retain representative examples selected by SemHash. The final cleaned and deduplicated dataset contains **8.5K verified query–command pairs** (30.7% of the original 27.7K generated data), forming a high-quality and executable benchmark for NL-to-CLI evaluation. We then sample from this dataset to construct an evaluation set of 5,000 examples (covering all 1,642 tools) and a training set of 3,504 examples (covering 962 tools), prioritizing full tool coverage in the evaluation set while reserving a smaller training split for studying generalization.

3. Evaluation Protocol. For each query in the evaluation set, models are prompted to generate an executable Kali Linux CLI command under three settings that vary the amount of tool information provided: (1) *Unrestricted*, where the model receives only the natural-language query and must infer both the appropriate tool and its arguments; (2) *Restricted*, where the model is additionally provided with a randomly sampled list of 20 candidate tool names that includes the correct tool, reducing the tool-selection search space while still requiring correct argument construction; and (3) *Hinted*, where the model further receives the official documentation of the candidate tools, approximating schema-based function-calling benchmarks in which tool functionality and parameters are explicitly specified. Details of the structured prompting format and prompt templates are provided in Appendix C.3.

Command Parsing. Model outputs are parsed using Python’s `shlex` library for lexical analysis of shell-style syntax, enabling faithful emulation of Kali Linux command parsing under UNIX conventions without executing commands. Because KaliBench explicitly records the ground-truth tool_name, optional arguments (as alias-aware flag–value pairs), and ordered positional arguments for each example, we can perform deterministic, component-wise scoring. A CLI command follows the general form <tool_name> <args>, where arguments are either optional or positional. Optional arguments begin with `--` or `-` and may take values; their order is flexible and aliases are permitted. Positional arguments consist only of values and must appear in a fixed order. See Fig. 3 for illustration.

¹OpenAI Codex with GPT-5.4 Thinking.

Evaluation Metrics. We report a set of fine-grained, component-level metrics, including *Tool Accuracy* (correct tool selection), *Optional-Argument F1* (alias-aware matching of optional flag names), *Positional-Argument F1* (multiset matching of positional argument values), *Total Score* (mean of tool and argument scores), and *Exact Correct*, which equals 1 only when the tool and all arguments exactly match the ground truth. These metrics enable detailed analysis of model performance across tool selection and argument construction. Details of the metric definitions and calculations are provided in Appendix L.

Capability Taxonomy and Security-Phase Annotation. To enable structured, domain-aware analysis beyond raw command correctness, we leverage capability metadata derived from Kali Linux’s official `kali-tools` metapackage taxonomy, which organizes offensive and defensive tools into 23 functional capability dimensions spanning the full cybersecurity workflow. Inspired by the Lockheed Martin Cyber Kill Chain [13], we group these dimensions into five security phases: *Reconnaissance & Initial Access* (information-gathering, 802-11, rfid, bluetooth, sdr, social-engineering), *Vulnerability Analysis* (vulnerability, fuzzing, web, database, voip), *Exploitation & Payload Delivery* (exploitation, sniffing-spoofing, hardware, wireless), *Post-Exploitation & Lateral Movement* (post-exploitation, passwords, windows-resources, crypto-stego), and *Defensive Analysis & Reporting* (reverse-engineering, forensics, reporting, gpu). Although some capabilities span multiple stages, each dimension is assigned a primary functional role for clarity, enabling aggregation of evaluation metrics across coherent capability dimensions and security phases and supporting systematic analysis of model performance in natural-language-to-CLI translation across different cybersecurity contexts.

4. Training Protocol. Our training data support LLM post-training via Supervised Fine-Tuning (SFT) and Reinforcement Learning with Verifiable Rewards (RLVR). We adopt the same prompt template used during evaluation, with the query, a randomly sampled list of candidate tools, and tool documentation as inputs. We concatenate data from all three evaluation settings, resulting in $3,504 \times 3 = 10,512$ training samples. Additional details are provided in Appendix G.2.

SFT. We use the constructed prompt template and treat the ground-truth command as the target output. The model is trained to generate the correct command given the query and optional tool information.

RLVR. We extend the prompt template by instructing the model to produce intermediate reasoning within `<think>...</think>` tags before generating the final command, following [9]. Our reward function is derived from component-level evaluation metrics, providing fully verifiable signals for accurate prediction of tool name, positional arguments, and alias-aware optional arguments. This enables fine-grained learning signals beyond a binary exact-match objective. An additional reward is assigned for exact command matches. Because ground-truth commands are validated during data construction via sandboxed execution, rewards can be computed without executing model outputs, enabling efficient RL without runtime. We include a formatting reward to enforce output structure.

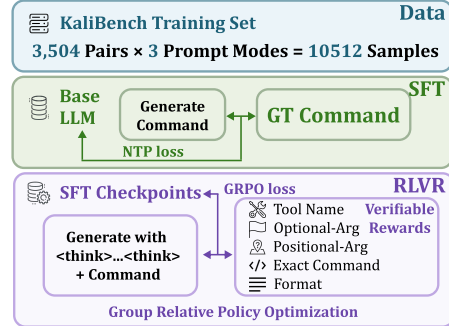


Figure 4: KaliBench Training Protocol.

4 Experiments

Experiment Setup. All evaluations of open-source models were conducted on an NVIDIA A100 multi-GPU cluster using the vLLM inference engine, with selected large models evaluated via external APIs². We evaluate 14 general-purpose models: *Llama3.1-Instruct (8B)* [8], *Qwen-3 (8B, 32B; with and without reasoning)* [41], *Mistral-3.2 (24B)* [23], *Gemma-3 (27B)* [35], *GPT-OSS (20B, 120B)* [27], *LLaMA-3.3-Instruct (70B)* [22], *Qwen-2.5-Instruct (72B)* [29], *Qwen3-Coder-Next (80B)* [4], and *DeepSeek-V3.2 (685B)* [6], *GLM-5.2 (753B)* [7] along with 7 cybersecurity-oriented models: *Foundation-Sec-Instruct (8B)* [39], *Foundation-Sec-Reasoning (8B)* [42], *Llama-Primus-Base/Merged (8B)* and *Llama-Primus-Nemotron (70B)* [43], and *RedSage-Ins/DPO (8B)* [34], as well as three variants of a cybersecurity-oriented model further fine-tuned on KaliBench. Models with

²OpenRouter: <https://openrouter.ai/>

Table 1: **Open-weight model query-to-command performance across three modes.** Columns report Unrestricted (U), Restricted (R), and Hinted (H). [†] indicates thinking-enabled models, and * denotes cybersecurity-tuned models. **Bold/underline** mark the best and second-best within each size group. Models are sorted by Averaged Total Score. MoE parameters use Total/Active (e.g., 685/37B).

Model	Size (B)	Exact Correct (%)			Tool Acc. (%)			Optional F1 (%)			Positional F1 (%)			Total Score (%)			
		U	R	H	U	R	H	U	R	H	U	R	H	U	R	H	Avg.
Compact Models (≤ 8B)																	
Foundation-Sec-Ins*	8	14.7	19.1	56.2	66.5	92.5	86.8	33.5	37.4	75.6	53.0	60.0	81.3	51.0	63.3	81.2	65.2
Llama3.1-Ins	8	12.5	18.2	62.7	61.7	93.7	95.4	32.4	37.2	82.7	53.5	61.9	85.8	49.2	64.3	88.0	67.2
Llama-Primus-Base*	8	15.8	19.3	60.3	67.3	90.7	94.1	36.6	39.2	80.8	56.3	61.9	84.0	53.4	63.9	86.3	67.9
Foundation-Sec-Rsn* [†]	8	13.3	18.6	66.6	67.0	92.8	95.9	34.8	39.1	85.5	53.0	57.9	86.2	51.6	63.3	89.2	68.0
Llama-Primus-Merged*	8	16.3	21.0	65.2	66.2	93.4	95.2	36.7	40.8	84.1	56.6	63.2	87.1	53.2	65.8	88.8	69.3
RedSage-DPO*	8	18.2	25.4	68.1	66.8	95.1	95.7	40.1	46.1	85.4	56.7	65.9	88.5	54.5	69.0	89.9	71.1
Qwen3	8	18.3	24.1	73.1	68.9	95.6	<u>96.3</u>	39.3	43.4	88.2	58.5	65.6	90.5	55.5	68.2	91.6	71.8
Qwen3 [†]	8	20.9	27.7	80.9	70.3	<u>95.7</u>	96.2	42.9	47.2	92.4	59.9	66.5	93.4	57.7	69.8	94.0	73.8
RedSage-Ins*	8	19.7	26.1	67.2	67.9	94.6	95.3	42.4	46.9	85.0	58.8	66.4	88.3	56.4	69.3	89.5	71.7
RedSage-K (GRPO)*[†]	8	28.1	33.0	66.7	76.5	93.3	93.1	51.7	53.7	84.8	73.9	77.0	88.4	67.4	74.6	88.8	76.9
RedSage-K (SFT)*	8	30.6	34.4	<u>77.1</u>	78.1	95.9	97.1	<u>52.8</u>	<u>55.2</u>	<u>90.5</u>	66.9	68.4	<u>92.0</u>	65.9	73.2	<u>93.2</u>	<u>77.4</u>
RedSage-K (SFT+GRPO)*[†]	8	32.2	37.4	69.4	<u>77.9</u>	95.0	92.5	<u>56.1</u>	<u>57.9</u>	86.5	76.9	80.9	89.1	70.3	77.9	89.3	79.2
Mid-size Models (20-32B)																	
Gemma-3-it	27	18.9	24.0	70.5	<u>73.4</u>	97.3	87.0	40.9	44.2	83.1	60.2	66.3	88.4	58.1	69.2	86.2	71.2
GPT-OSS [†]	20/3.6	19.6	24.6	78.1	72.3	95.1	95.9	42.2	45.6	91.3	57.1	63.6	92.0	57.2	68.1	93.0	72.8
Mistral-Small-3.2	24	<u>20.7</u>	27.4	79.9	70.6	95.6	96.2	<u>44.3</u>	48.2	<u>91.5</u>	62.6	69.2	93.2	59.2	71.0	<u>93.7</u>	74.6
Qwen3	32	20.6	<u>27.7</u>	79.8	73.5	96.3	95.5	43.4	<u>48.9</u>	91.2	<u>62.8</u>	<u>69.9</u>	<u>93.4</u>	<u>59.9</u>	<u>71.7</u>	<u>93.4</u>	<u>75.0</u>
Qwen3 [†]	32	24.0	30.9	81.8	72.4	<u>96.8</u>	95.2	47.2	51.6	92.4	65.3	71.4	94.0	61.6	73.3	93.9	76.3
Large-scale Models (≥ 70B)																	
Llama-Primus-Ins*	70	18.8	25.0	71.2	72.8	96.4	92.1	41.9	46.3	88.1	59.6	67.0	90.2	58.1	69.9	90.1	72.7
Llama-3.3-Ins	70	22.6	28.7	77.7	72.1	96.5	94.1	45.7	49.9	91.7	61.9	68.1	92.5	59.9	71.5	92.7	74.7
GPT-OSS [†]	120/5.1	24.0	32.0	<u>80.9</u>	73.6	95.6	96.0	48.0	52.9	<u>92.2</u>	61.4	68.7	92.7	61.0	72.4	93.6	75.7
Qwen3-Coder-Next [†]	80/3	26.2	31.0	79.9	74.1	96.8	96.4	48.8	51.9	91.4	63.9	70.7	93.2	62.3	73.1	93.6	76.3
Qwen2.5-Ins	72	25.4	30.2	80.6	<u>76.0</u>	<u>97.4</u>	<u>96.8</u>	47.7	51.1	91.5	<u>67.0</u>	72.2	<u>93.3</u>	63.6	73.6	<u>93.8</u>	77.0
DeepSeek-V3.2 [†]	685/37	<u>33.0</u>	<u>40.2</u>	84.5	75.8	98.3	97.1	<u>57.0</u>	<u>62.0</u>	93.5	68.8	<u>75.4</u>	94.1	<u>67.2</u>	<u>78.6</u>	94.9	<u>80.2</u>
GLM-5.2 [†]	753	41.3	52.0	74.8	86.3	94.2	94.9	75.3	71.9	88.4	51.3	78.8	<u>90.6</u>	71.0	81.7	91.3	81.3
Metric Average	—	22.3	28.3	73.1	72.0	95.2	94.6	45.1	48.7	87.8	61.1	68.2	90.1	59.4	70.7	90.8	73.6

72B or more parameters were evaluated through the OpenRouter API; all other models are obtained from their official Hugging Face repositories under their respective licenses and evaluated under a fixed inference configuration; details are in Appendix G.1.

Training Setup. We employ parameter-efficient fine-tuning using LoRA [12] with Unsloth [5]. We select the best-performing 8B cybersecurity-tuned model *RedSage-Ins* as the backbone. Using the training dataset described in Sec. 3, we train three variants, collectively referred to as *RedSage-K*: (1) SFT, (2) RLVR with Group Relative Policy Optimization (GRPO) [33], and (3) SFT followed by GRPO (SFT+GRPO). All models are trained on a single H200 GPU (141GB VRAM) with a total batch size of 32 for two epochs; details are in Appendix G.2.

4.1 Model Performance Across Three Modes

Table 1 summarizes per-model performance under three modes: *Unrestricted* (*U*), *Restricted* (*R*), and *Hinted* (*H*). We report Exact Correct (EC), Tool Accuracy (TA), Optional-Argument F1 (Opt-F1), Positional-Argument F1 (Pos-F1), Total Score (TS), and Average TS (Avg.). We analyze key patterns with respect to tool information, model type and scale, and post-training on KaliBench.

Impact of Evaluation Modes. The three evaluation modes reveal a clear separation between tool selection and argument construction. Transitioning from *U* to *R* substantially improves TA (72.0%→95.2% on average), indicating that constraining the candidate tool space largely resolves tool-selection errors. However, this shift yields only modest gains in Opt-F1, Pos-F1, and EC, indicating that correct tool selection alone does not ensure accurate argument generation. The metric averages further reveal a hierarchy of difficulty: optional-argument prediction is the most challenging, followed by positional-argument prediction. In contrast, *H* produces the largest improvements, increasing Opt-F1 from 45.1% to 87.8%, Pos-F1 from 61.1% to 90.1%, TS from 59.4% to 90.8%, and EC from 22.3% to 73.1%. These gains indicate that tool hints resolve ambiguities in argument selection, ordering, and formatting, and suggest that existing models lack sufficient domain knowledge to construct accurate commands without tool documentation. Overall, *R* primarily improves *which* tool to use, whereas *H* improves *how* the command is constructed; without hints, open-weight models still struggle with accurate argument generation, the primary bottleneck.

Impact of Model Type and Scale. Cybersecurity-oriented models (*) do not consistently outperform strong general-purpose models on query-to-command generation, but their advantages become more apparent in less-guided settings. In the compact regime, *RedSage-Ins** achieves higher TS than non-thinking *Qwen3* in both *U* and *R* (56.4% vs. 55.5% and 69.3% vs. 68.2%), suggesting that domain-specific training provides useful priors when models must infer command structure with limited support. In contrast, general-purpose models perform better in *H*, where *Qwen3* surpasses *RedSage-Ins** (91.6% vs. 89.5%), indicating stronger instruction-following and context utilization under richer guidance. Overall, cybersecurity-oriented models are more effective in under-specified settings, while general-purpose models better exploit explicit hints. Reasoning-enabled variants (†) further improve argument-level performance, particularly in Opt-F1, Pos-F1, and EC. For example, *Qwen3†* improves Avg from 71.8% to 73.8% over *Qwen3* and achieves strong *H*-mode results (94.0% TS, 80.9% EC), highlighting the role of reasoning in intent decomposition and context utilization. Finally, while performance generally improves with scale, gains are uneven. For example, *Qwen3†* (8B) outperforms larger models, including *Gemma-3-it* (27B) and *Llama-Primus-Ins** (70B), in Avg. These results suggest that effective NL-to-CLI generation depends on domain knowledge, instruction-following, reasoning, and task alignment, rather than scale alone.

Proprietary Models and Scaffolded Inference. Our main evaluation emphasizes open-weight models because local deployment is particularly important in cybersecurity, where queries, target information, and operational data may be sensitive and cannot always be shared with proprietary APIs. To assess whether KaliBench remains challenging for stronger proprietary models and scaffolded systems, we additionally evaluate GPT-5.6-Sol [26], Claude Opus 5 [2], and OpenAI Codex [24], powered by GPT-5.5 [25] with xhigh reasoning effort. We evaluate all systems on the 5,000 test queries in the *Unrestricted* setting.

As shown in Table 2, GPT-5.6-Sol achieves the highest full-set Exact Correct of 61.68%, followed by Codex at 51.68% and Claude Opus 5 at 44.02%. All three outperform GLM-5.2, the strongest evaluated open-weight model in this setting, which achieves 41.32%. Response coverage, however, varies across systems: GPT-5.6-Sol answers 4,988 of 5,000 queries, compared with 4,633 for Codex and 3,675 for Claude Opus 5. These coverage differences suggest that provider-specific safety restrictions influence cybersecurity-related usage. For example, Claude Opus 5 achieves 59.89% EC on answered queries but answers only 73.50% of the test set, reducing its full-set EC to 44.02%. Moreover, despite the gains over open-weight models, the highest full-set accuracy among these systems is 61.68%, indicating that KaliBench remains unsaturated and that exact command generation is challenging. Complete metrics and reporting details are provided in Appendix M.

Table 2: Proprietary models and scaffolded inference in Unrestricted mode.

Model	Answered queries	EC (%)	EC (%) (all 5K)
GPT-5.6-Sol	4,988	61.83	61.68
Claude Opus 5	3,675	59.89	44.02
Codex (GPT-5.5)	4,633	55.77	51.68

EC (Exact Correct) is computed over answered queries; the EC (all 5K) counts blocked or missing outputs as incorrect. For Codex, we use its scaffolded CLI workflow and score only the final submitted command.

Impact of KaliBench Post-training. Post-training on the KaliBench training split substantially improves compact models. The three *RedSage-K* variants achieve Avg scores of 76.9, 77.4, and 79.2, with *RedSage-K (SFT+GRPO)* ranking third in Avg among all evaluated open-weight models, only 1.0 point below *DeepSeek-V3.2†* despite using an 8B backbone. The methods exhibit complementary strengths: *RedSage-K (SFT)* attains the best compact-model TA across all modes, shows stronger gains in Opt-F1, and delivers the largest improvements in *H*, suggesting that supervised fine-tuning effectively learns tool mapping and benefits from contextual hints. In contrast, *RedSage-K (GRPO)* shows stronger gains in Pos-F1 and TS under *U* and *R*, but is weaker in *H*.

Combining both methods, *RedSage-K (SFT+GRPO)* yields the best compact-model *U/R* TS and EC, demonstrating the benefit of combining SFT and RLVR under limited guidance. However, *RedSage-K (GRPO)* and *RedSage-K (SFT+GRPO)* slightly reduce *H*-mode TS relative to the original *RedSage-Ins* model. Our paired analysis identifies argument-construction errors and regressions on previously solved examples, with SFT initialization mitigating these regressions (Appendix I). Overall, KaliBench post-training enables compact models to approach larger ones in cybersecurity tool use under less-guided settings.

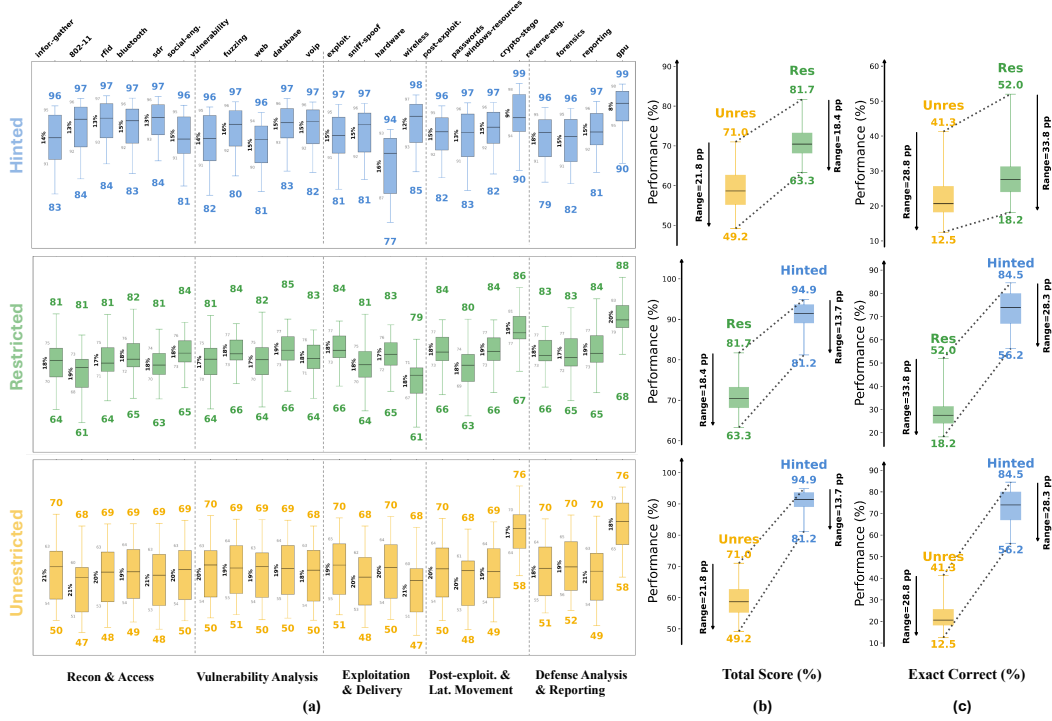


Figure 5: **Performance across security dimensions and modes.** (a) Performance over 23 dimensions and 5 phases per mode. Box plots show distributions (Q1–Q3), with whiskers for min/max. (b,c) Total Score and Exact Correct (%) across modes; dotted lines indicate inter-mode gaps.

4.2 Fine-Grained Performance and Robustness Analysis

Dimension-wise and Tool-level Analysis. Figure 5(a) reports TS performance across 23 capability dimensions, five security phases, and three evaluation modes, aggregated over all evaluated models. Performance varies across dimensions, but relative difficulty remains broadly consistent across modes, suggesting that additional tool information improves overall scores without fundamentally changing which dimensions are harder. Dimensions such as *gpu* and *crypto-stego* consistently achieve higher scores, whereas *802-11* and *wireless* show lower minima, especially in *U* and *R* modes. We also observe mode-specific deviations, such as *hardware* in *H* mode, where performance remains comparatively lower, likely reflecting the difficulty of longer or more complex tool descriptions.

From a distributional perspective, variability decreases as tool information increases. The *U* mode shows the widest spread across models, reflecting high variance without tool guidance; *R* reduces this variability by constraining tool selection; and *H* further stabilizes performance across most dimensions. However, *H* narrows variance without eliminating dimension-level gaps, suggesting that documentation improves argument construction but does not fully resolve interface complexity. At the phase level, Reconnaissance and Vulnerability Analysis are relatively stable, while Post-Exploitation and Defensive Analysis show greater variation, likely due to more heterogeneous CLI structures.

At the tool level, performance differences largely reflect CLI interface complexity. Tools with simple invocation patterns and well-defined defaults (e.g., *truecrypt2john*, *jsql*) are handled reliably, whereas those with irregular or uncommon options (e.g., *nikto*, *hashrat*) remain challenging due to precise flag–value binding and strict syntax. Additional full statistics are provided in Appendix K, confirming that tool-level difficulty is driven more by interface complexity than model-specific limitations.

Mode-wise Performance Trends. Figure 5(b,c) visualizes the distributions of Total Score and Exact Correct across the three evaluation modes. Both metrics improve consistently from *U* to *R* to *H*, with *H* achieving the strongest results. This supports our earlier finding that tool documentation provides the largest gains, primarily by improving argument prediction, the main bottleneck in accurate command construction.

The two metrics exhibit different patterns of cross-model dispersion. The Total Score range narrows from 21.8 $\rightarrow$ 18.4 $\rightarrow$ 13.7 percentage points across U , R , and H . In contrast, the Exact Correct range remains wide at 28.8, 33.8, and 28.3 percentage points, respectively. This suggests that providing tool information brings models closer together in Total Score, while substantial differences in exact command correctness persist.

Common Failure Pattern Analysis. Weak query-to-command performance primarily stems from accumulated errors in command construction, particularly with optional arguments. Across models, we observe four recurring patterns: (i) *long-short option misuse*, where alias forms are mixed (e.g., `--domain/-d`), producing invalid commands; (ii) *option hallucination*, where positional arguments are treated as flags or nonexistent aliases inferred (e.g., `'st --client'`); (iii) *hallucinated long options*, where tools with only short flags lead to invented long forms; and (iv) *erroneous option names*, where singular-plural confusion produces incorrect variants (e.g., `--symlink` vs. `--symlinks`).

Robustness to Query Variation. Since KaliBench queries are synthesized from tool documentation, we examine the sensitivity of model performance to changes in query wording and presentation. We evaluate eight model configurations under three query variations: *Paraphrase* (uses synonyms and alternative phrasing); *Informal* (adopts conversational wording); and *Messy* (introduces linguistic and formatting noise, such as grammatical errors, omitted prepositions or verbs, and incorrect word choices).

Table 3: Average change in Total Score, in percentage points, relative to original queries across eight model configurations.

Query style	Ures.	Res.	Hinted
Paraphrase	+0.61	+0.22	-0.09
Informal	-0.55	-0.48	-0.28
Messy	-3.20	-1.40	-1.07

Table 3 shows that paraphrases have little overall effect and slightly improve performance in the *Unrestricted* and *Restricted* settings. Informal queries introduce a small but consistent degradation. Messy queries cause the largest drop across all settings, especially in the *Unrestricted* setting. *Hinted* mode shows the smallest decline under messy inputs (1.07 percentage points), suggesting that access to full tool documentation mitigates, but does not eliminate, the impact of noisy wording. The eight configurations and their model-level results for the *Messy* variation are detailed in Appendix N.

5 Discussion and Conclusion

KaliBench exposes a clear gap between LLMs’ general cybersecurity knowledge and their ability to generate exact, executable CLI commands. Our results show that tool selection and argument construction are fundamentally different challenges: restricting candidate tools largely resolves tool-selection errors, while argument-level correctness, especially for optional flags and flag-value bindings, remains the primary bottleneck. Providing tool documentation results in the largest gains, indicating that failures often stem from limited domain knowledge rather than intent understanding. These findings highlight the importance of evaluating tool use at the command-structure level in schema-free settings, where models must infer tools and syntax without explicit APIs.

Beyond evaluation, KaliBench enables effective model improvement through fine-grained, runtime-free verifiable rewards. Post-training with SFT and RLVR substantially improves compact models, allowing an 8B model to approach much larger systems. Despite these advances, limitations remain, including the focus on single-turn command generation and reliance on documentation-grounded data. Future work should extend to multi-step workflows, retrieval-augmented tool use, and environment-aware evaluation. Overall, KaliBench provides a practical benchmarking and training framework for advancing the reliable and executable use of LLM-based cybersecurity tools.

6 Acknowledgements

This research was funded by Khalifa University of Science and Technology through the Faculty Start-Ups under Project ID: KU-INT-FSU-2005-8474000775. This research was also supported by KU Research Center for Advanced Intelligent Systems (AIS), Khalifa University of Science and Technology (KU-AIS).

References

- [1] Md Tanvirul Alam, Dipkamal Bhusal, Le Nguyen, and Nidhi Rastogi. Ctibench: A benchmark for evaluating llms in cyber threat intelligence. *Advances in Neural Information Processing Systems*, 37:50805–50825, 2024.
- [2] Anthropic. System Card: Claude Opus 5. System card, Anthropic, 2026. URL <https://www.anthropic.com/claude-opus-5-system-card>.
- [3] Dipkamal Bhusal, Md Tanvirul Alam, Le Nguyen, Ashim Mahara, Zachary Lightcap, Rodney Frazier, Romy Fieblinger, Grace Long Torales, Benjamin A Blakely, and Nidhi Rastogi. Secure: Benchmarking large language models for cybersecurity. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 15–30. IEEE, 2024.
- [4] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. Qwen3-coder-next technical report, 2026. URL <https://arxiv.org/abs/2603.00729>.
- [5] Michael Han Daniel Han and Unsloth team. Unsloth, 2023. URL <https://github.com/unslothai/unsloth>.
- [6] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y. Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yudian Wang, Yue Gong, Yuhang Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, R. L. Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojuan Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T. Wang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, and Zihua Qu. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.

- [7] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezheng Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang,

- and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [10] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11143–11156, 2024.
 - [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
 - [12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
 - [13] Eric M Hutchins, Michael J Cloppert, Rohan M Amin, et al. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1):80, 2011.
 - [14] Pengfei Jing, Mengyun Tang, Xiaorong Shi, Xing Zheng, Sen Nie, Shi Wu, Yong Yang, and Xiapu Luo. Secbench: A comprehensive multi-dimensional benchmarking dataset for llms in cybersecurity. *arXiv preprint arXiv:2412.20787*, 2024.
 - [15] Kali Linux Team. kali-tools: Markdown format of www.kali.org/tools/, 2022. URL <https://gitlab.com/kalilinux/documentation/kali-tools>. GitLab repository, accessed March 31, 2025.
 - [16] Guancheng Li, Yifeng Li, Wang Guannan, Haoyu Yang, and Yang Yu. Seceval: A comprehensive benchmark for evaluating cybersecurity knowledge of foundation models. <https://github.com/XuanwuAI/SecEval>, 2023.
 - [17] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
 - [18] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Alexander Cosgrove, Christopher D Manning, Christopher Re, Diana Acosta-Navas, Drew Arad Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue WANG, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri S. Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Andrew Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=i04LZibEqW>.
 - [19] Zefang Liu, Jialei Shi, and John F Buford. Cyberbench: A multi-task benchmark for evaluating large language models in cybersecurity. In *AAAI 2024 Workshop on Artificial Intelligence for Cyber Security*, 2024.
 - [20] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37: 54463–54482, 2024.
 - [21] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025.

- [22] Meta AI. Llama-3.3-70b-instruct. <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>, 2024. Model card, released December 6, 2024, accessed May 5, 2026.
- [23] Mistral AI. Mistral-small-3.2-24b-instruct-2506. <https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506>, 2025. Model card, accessed May 5, 2026.
- [24] OpenAI. Codex CLI, . URL <https://learn.chatgpt.com/docs/codex/cli>. Accessed: 2026-09-27.
- [25] OpenAI. GPT-5.5 model, . URL <https://developers.openai.com/api/docs/models/gpt-5.5>. Accessed: 2026-09-27.
- [26] OpenAI. GPT-5.6 Sol model, . URL <https://developers.openai.com/api/docs/models/gpt-5.6-sol>. Accessed: 2026-09-27.
- [27] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrlyov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- [28] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [29] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [30] Qwen Team. Qwen3-max: Just scale it. <https://qwen.ai/blog?id=qwen3-max>, 2025. Blog post, accessed May 5, 2026.
- [31] Offensive Security. Kali tools, 2025. URL <https://www.kali.org/tools/>. Accessed: 2025-12-03.
- [32] Minghao Shao, Sofija Jancheska, Meet Udeshi, Brendan Dolan-Gavitt, Haoran Xi, Kimberly Milner, Boyuan Chen, Max Yin, Siddharth Garg, Prashanth Krishnamurthy, et al. Nyu ctf bench: A scalable open-source benchmark dataset for evaluating llms in offensive security. *Advances in Neural Information Processing Systems*, 37:57472–57498, 2024.

- [33] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [34] Naufal Suryanto, Muzammal Naseer, Pengfei Li, Syed Talal Wasim, Jinhui Yi, Juergen Gall, Paolo Ceravolo, and Ernesto Damiani. Redsage: A cybersecurity generalist llm. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=W4FAenIrQ2>.
- [35] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Pöder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- [36] Norbert Tihanyi, Mohamed Amine Ferrag, Ridhi Jain, Tamas Bisztray, and Merouane Debbah. Cybermetric: A benchmark dataset based on retrieval-augmented generation for evaluating llms in cybersecurity knowledge. In *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 296–302. IEEE, 2024.
- [37] Thomas van Dongen and Stephan Tulkens. Semhash: Fast multimodal semantic deduplication & filtering, 2025. URL <https://github.com/MinishLab/semhash>.
- [38] Jun Wang, Jiamu Zhou, Xihuai Wang, Xiaoyun Mo, Haoyu Zhang, Qiqiang Lin, Cheng Jin, Muning Wen, Weinan Zhang, Qiuying Peng, and Jun Wang. HammerBench: Fine-grained function-calling evaluation in real mobile assistant scenarios. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3350–3376, Vienna, Austria, July 2025. Association for

- Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.175. URL <https://aclanthology.org/2025.findings-acl.175/>.
- [39] Sajana Weerawardhena, Paul Kassianik, Blaine Nelson, Baturay Saglam, Anu Vellore, Aman Priyanshu, Supriti Vijay, Massimo Aufiero, Arthur Goldblatt, Fraser Burch, et al. Llama-3.1-foundationai-securityllm-8b-instruct technical report. *arXiv preprint arXiv:2508.01059*, 2025.
 - [40] HanXiang Xu, ShenAo Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Kai Chen, Ting Yu, Yang Liu, and HaoYu Wang. Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2025.
 - [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
 - [42] Zhuoran Yang, Ed Li, Jianliang He, Aman Priyanshu, Baturay Saglam, Paul Kassianik, Sajana Weerawardhena, Anu Vellore, Blaine Nelson, Neusha Javidnia, Arthur Goldblatt, Fraser Burch, Avi Zohary, Assaf Eisenman, Mahdi Sabbaghi, Supriti Vijay, Rahim Dharssi, Dhruv Kedia, Kojin Oshiba, Yaron Singer, and Amin Karbasi. Llama-3.1-foundationai-securityllm-reasoning-8b technical report, 2026. URL <https://arxiv.org/abs/2601.21051>.
 - [43] Yao-Ching Yu, Tsun-Han Chiang, Cheng-Wei Tsai, Chien-Ming Huang, and Wen-Kwang Tsao. Primus: A pioneering collection of open-source datasets for cybersecurity LLM training. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10391–10413, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.527. URL <https://aclanthology.org/2025.emnlp-main.527/>.
 - [44] Zhengmin Yu, Jiutian Zeng, Siyi Chen, Wenhan Xu, Dandan Xu, Xiangyu Liu, Zonghao Ying, Nan Wang, Yuan Zhang, and Min Yang. Cs-eval: A comprehensive large language model benchmark for cybersecurity. *arXiv preprint arXiv:2411.16239*, 2024.
 - [45] Andy K Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Julian Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Kenny O Oseleononmen, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tc90LV0yRL>.
 - [46] Chiyu Zhang, Marc-Alexandre Cote, Michael Albada, Anush Sankaran, Jack W Stokes, Tong Wang, Amir Abdi, William Blum, and Muhammad Abdul-Mageed. Defenderbench: A toolkit for evaluating language agents in cybersecurity environments. *arXiv preprint arXiv:2506.00739*, 2025.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: Our paper’s **Abstract** and **Section 1: Introduction** clearly define the article’s contributions and scope.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We carefully discuss the limitations of our work in **Appendix O: Limitations**.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory Assumptions and Proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: The contributions of this paper focus on experimental results rather than theoretical derivations and theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We disclose the full code and data in the link of our project page (**Abstract**). In addition, the experiments in this paper evaluate multiple open-source models and utilize the vLLM engine during the inference phase; these settings ensure the standardization and reproducibility of the experiments (**Section 4: Experiments**). More specifically, **Appendix B: CLI Command Generation** and **Appendix C: Prompt Templates** provide all the prompts needed to reproduce the generation of the benchmark data, while **Appendix E: Terminal verification Details** provides the rules for sample-wise terminal verification, and **Appendix L: Details about Metrics Calculation** details the metric calculations required to accurately reproduce the result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide full access to the data (via Hugging Face) and the complete implementation code (via GitHub). Link contains these resources can be found at the beginning of the paper, along with sufficient instructions to reproduce the main experimental results.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The dataset splits, evaluation protocol, and training methods are described in **Section 3**. Model configurations and detailed training hyperparameters are provided in **Appendix G**.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper provides information about variability through distributional statistics. In particular, **Figure 5** reports box plots (Q1–Q3 with min/max whiskers) across all models, security dimensions, and mode settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We have provided the GPU model, memory, and the amount of compute required for the experiment (**Section 4: Experiments, Appendix G.2**). The full research project required the same compute as reported in the paper.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code Of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We carefully checked the NeurIPS Code of Ethics and discussed in **Appendix A: Ethical Statement** that this paper does not raise any ethical issues.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader Impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [\[Yes\]](#)

Justification: We discuss the potential societal impacts in **Appendix A: Ethical Statement**.

Guidelines:

- The answer [\[N/A\]](#) means that there is no societal impact of the work performed.
- If the authors answer [\[N/A\]](#) or [\[No\]](#), they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [\[Yes\]](#)

Justification: The paper discusses potential dual-use risks of the cybersecurity benchmark and describes several release-related precautions in **Appendix A: Ethical Statement**.

Guidelines:

- The answer [\[N/A\]](#) means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: **Appendix A: Ethical Statement** documents the existing assets used in this work, credits their original sources, and states the applicable licenses and terms of use; we listed the evaluated model details in **Appendix G.1**.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces KaliBench as a new benchmark asset, and **Appendix A: Ethical Statement** documents its construction, intended use, licensing, and release considerations. Besides, Section 3 describes the benchmark’s construction, data structure, verification, splits, and evaluation protocol.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: **Appendix A: Ethical Statement** states that no data were collected from human subjects and no external annotators were involved; the paper does not include crowdsourcing or human-subject research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: **Appendix A: Ethical Statement** states that no data were collected from human subjects and no external annotators were involved; therefore, this work does not involve human-subject research or crowdsourcing.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The paper clearly describes the use of LLMs as a core component, including their roles in data generation, verification (LLM-as-verifier), and model training (SFT and RL with verifiable rewards), as detailed in **Section 3**.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Ethical Statement

Existing Assets. All data used in this work are derived from publicly available Kali Linux tool manuals and documentation. This source repository is part of the Kali Linux documentation project and is licensed under the GNU General Public License v3.0. Model inference was performed either offline in a controlled environment or via APIs. Open-weight models and proprietary API services are subject to their respective model licenses and provider terms. Except for our KaliBench post-trained variants, we did not perform additional fine-tuning on KaliBench.

New Assets. KaliBench is intended for research purposes only and is released under the CC BY-NC 4.0 license. KaliBench is released together with documentation describing the structure of the benchmark, dataset fields, data construction and verification pipeline, train–test splits, evaluation protocol, licensing terms, intended use, limitations, and responsible-use guidelines. Since the dataset is derived from publicly available Kali Linux documentation and contains no personally identifiable information or human-subject data, no individual consent is required. We also provide release notes and usage instructions to support reproducible and responsible use of the benchmark.

Artifact availability. The public repository provides code and instructions for data generation, filtering, verification, evaluation, and post-training, including SFT and GRPO.³ Croissant metadata was also provided with the OpenReview submission to support machine-readable dataset documentation.

Human Subjects. The KaliBench dataset contains no personally identifiable information. No data were collected from human subjects, and no external annotators were involved in data creation or evaluation. All human-in-the-loop verification is conducted by the authors solely for internal data quality control; no external annotators, crowd workers, or human subjects were involved.

Social Impact. The main positive societal impact of this work lies in improving the reliability evaluation of large language models in cybersecurity settings. This work helps reduce the risk that security-oriented LLMs generate incorrect commands or mislead users before real-world deployment, and can support the development of more reliable defensive tools for legitimate scenarios such as security education, vulnerability assessment, incident response, and digital forensics. This work does not introduce new attack techniques, vulnerability exploitation methods, or end-to-end attack automation workflows. The data are primarily derived from publicly available Kali Linux tool documentation. KaliBench focuses only on single-command-level translation and evaluation, and does not provide multi-step attack chains, target-environment-adaptive attack procedures, or end-to-end automated penetration capabilities. Therefore, we believe that KaliBench does not directly introduce new security threats.

LLM Usage. LLMs were used for query–command generation, documentation-grounded verification, and preliminary assessment of suspicious examples, with final manual-review decisions made by the authors. LLMs were also used for language editing of the manuscript. All experimental design, analysis, and conclusions were produced and verified by the authors.

B CLI Command Generation

KaliBench focuses on deterministic *natural-language-to-CLI* translation at scale within Kali Linux (~2,372 tools). It defines reviewer-checkable canonicalization rules that specify:

- i. **Ordering:** the command begins with the tool name, followed by all long flags, then all short flags (lexicographically), then flags with values, and finally positional arguments in document order.
- ii. **Flag syntax:** flags and their values are space-separated (e.g., ‘--opt value’, ‘-s value’). Equals-style bindings (e.g., ‘--opt=value’) are allowed. Boolean flags take no value.
- iii. **Quoting and aliasing:** quoting is applied only when required (e.g., values containing whitespace or shell meta-characters). Aliases (e.g., ‘--help’ vs. ‘-h’) are normalized, but the model must output a single canonical form—preferably the long-flag variant.

To ensure unambiguous extraction, all CLI predictions are enclosed within a standardized `<output>` `</output>` tag. This enables robust isolation and symbolic comparison of the core command.

In all, these constraints enable scalable and accurate evaluation by supporting diverse arguments and alias combinations in the data generation process, ensuring reproducibility through exact parse-tree matching.

C Prompt Templates

In this section, we list the three prompt templates used in our study, corresponding to the query-command generation, LLM-assisted data verification, and model evaluation.

C.1 Prompt for Generating the Query-Command Pairs

This prompt defines a controlled data generation setting for constructing a cybersecurity command benchmark from official Kali/Linux tool documentation. The model is instructed to synthesize

³<https://github.com/RISys-Lab/KaliBench>

natural-language queries that uniquely determine a single, canonical shell command, together with structured argument annotations. Strict constraints enforce faithfulness to documented tool capabilities, deterministic command formatting, and explicit parameter specification, thereby reducing ambiguity and unsupported behaviors. The resulting dataset enables reliable evaluation of command generation, argument parsing, and hallucination under reproducible conditions.

System Instruction

“You are generating a cybersecurity tool benchmark dataset. Your task is to convert seed documentation of Kali/Linux tools into a structured JSON dataset...

Goal

The goal is to generate a JSON array where each entry consists of: 1.An unambiguous natural language query 2.The exact ground-truth shell command 3.The canonical tool name 4.Parsed optional arguments (flag–value mapping) 5.Ordered positional arguments.

Hard Constraints

1. Use only tools, flags, and behaviors explicitly documented in the seed.
2. Commands must follow deterministic canonicalization (tool → long flags → short flags → value flags → positional arguments).
3. No invented flags, pipelines, subshells, or implicit defaults.
4. Each query must admit exactly 1 valid command, with aliases documented.

Unambiguity Enforcement

To ensure a single correct answer:

- All required parameters must be explicit; vague phrasing (e.g., “quick”, “default”) is forbidden.
- A uniqueness self-check is required (no alternative command fits the query).

Output Format

The output must be a JSON array, where each item strictly follows:

```
{
  "query": "string",
  "ground_truth_command": "string",
  "tool_name": "string",
  "optional_args": { "flag|alias":
    "value or null" },
  "positional_args": ["arg1", "arg2"]
}
```

No additional text is permitted.

C.2 Prompt for LLM-assisted Data Verification

The following prompt is designed for dataset cleaning and validation by instructing the model to act as a strict CLI command verifier. Given a <natural language query>, a <corresponding ground-truth command>, and <the official usage documentation of a specified tool>, the model evaluates whether the command is valid or hallucinated. The verification focuses on flag existence, argument correctness, and functional alignment with the documented capabilities of the tool. The output consists of a binary verdict accompanied by a concise justification, enabling reliable filtering of erroneous or unsupported command annotations.

System Instruction

“You are a meticulous CLI command verifier. Verify whether the query–ground truth pairs (query, ground_truth_command) are accurate or hallucinated based on the provided <Tool Documentation>.”

Verification Criteria

The model must carefully check:

1. Whether every flag and argument in the ground-truth command appears in the tool usage documentation and is used correctly.
2. Whether the command can reasonably implement the requested query using only the capabilities described in the usage documentation.
3. If any flag does not exist or the requested behavior is unsupported, the command should be classified as hallucinated.

Output Format

The model must reason step-by-step and produce the following structured output:

```
### Reasoning
<Step-by-step reasoning>

### Output JSON
{
  "verdict": "ACCURATE"/"HALLUCINATED",
  "reasons": "<short explanation>"
}
```

C.3 Prompt for Model Evaluation

The following prompt tells the model how to analyze a query and generate corresponding CLI commands in **Unrestricted** mode for **evaluation**. The model is instructed to act as a cybersecurity function-calling agent with direct access to a Kali/Linux execution environment. Given a user query, the model is required to output a single, executable shell command that precisely accomplishes the requested task. The prompt enforces strict constraints on tool selection, argument completeness, and option–value correctness, while explicitly prohibiting explanatory text or non-command output. This design enables controlled evaluation of the model’s accuracy and completeness in generating real-world security-related commands. System prompts for **Restricted** and **Hinted** modes are similar as that of unrestricted. The only difference is that these two modes embed a subset of <“a group of tool names”> and <“a group of tool names with their corresponding usage manuscripts”> in the Task Definition Part.

System Instruction

“You are a cybersecurity function-calling AI model. You have access to the run_terminal. Execute a shell command in a Kali/Linux terminal and return stdout, stderr, and exit code.”

Task Definition

“Given a <USER QUERY>, generate the single most accurate shell command using any appropriate Kali/Linux tool(s) to solve the query.”

Command Generation Constraints

1. Use the correct command-line tool(s) appropriate for the task.
2. Include all required optional arguments (flags beginning with - or --).
3. Correctly pair option keys and values (e.g., --port 80, -A INPUT).
4. Preserve all positional arguments (e.g., IP addresses, filenames, interfaces).
5. Do **not** invent flags or options that do not exist in real Kali/Linux tools.

1. Recon & Access	2. Vulnerability Analysis	3. Exploitation & Delivery ALIAS	4. Post-exploit. & Lateral Movement
(a) information-gathering Query: "Retrieve computer data from the domain 'corp.local' while resolving IP addresses of computer objects via mslookup on the domain controller." Tool_name: wpscan Optional_args: ["-enumerate", "p", "-url": "http://wordpress.local"] Positional_args: ["corp.local"] GT_cmd: impactet-GotADComputers --resolveIP corp.local	(a) vulnerability Query: "Enumerate all installed plugins on the WordPress site at http://wordpress.local" Tool_name: wpscan Optional_args: ["-enumerate", "p", "-url": "http://wordpress.local"] Positional_args: [] GT_cmd: wpscan --enumerate p --url http://wordpress.local	(a) exploitation Query: "Enumerate all installed plugins on the WordPress site at http://wordpress.local" Tool_name: wpscan Optional_args: ["-enumerate", "p", "-url": "http://wordpress.local"] Positional_args: [] GT_cmd: wpscan --enumerate p --url http://wordpress.local	(c) windows-resources Query: "Parse the Windows SAM registry hive to enumerate local user accounts for forensic analysis." Tool_name: regripper Optional_args: ["-a": "SAM", "-p": "sampire"] Positional_args: [] GT_cmd: regripper -a SAM -p sampire
(b) 802-11 Query: "List all wireless devices and their block status using rfkill with JSON output format." Tool_name: rfkill Optional_args: ["-json -j": null] Positional_args: ["list"] GT_cmd: rfkill -json list	(b) fuzzing Query: "Run llvms-reduce on 'crash.ll' with the interestingness test named 'repro.sh', passing '-arg1 value1' to the test script, and limit reductions to 2 threads." Tool_name: llvms-reduce Optional_args: ["-j": "2", "-test": "repro.sh", "-test-arg": "-arg1 value1"] Positional_args: ["crash.ll"] GT_cmd: llvms-reduce -j 2 --test repro.sh --test-arg '-arg1 value1' crash.ll	(b) sniffing-spoofing Query: "Use webspdy to monitor HTTP traffic from the host 192.168.1.100 using the default network interface." Tool_name: webspdy Optional_args: [] Positional_args: ["192.168.1.100"] GT_cmd: webspdy 192.168.1.100	(d) crypto-stego Query: "Extract encrypted hidden data from secret.jpg using steghide with the passphrase mypassword." Tool_name: steghide Optional_args: ["-sf": "secret.jpg", "-p": "mypassword"] Positional_args: ["extract"] GT_cmd: steghide extract -sf secret.jpg -p mypassword
(c) rfid Query: "Scan for high-frequency RFID tags and display their type and UID." Tool_name: proxmark3 Optional_args: ["-cmd": "hf search"] Positional_args: ["dev/ttyACM0"] GT_cmd: proxmark3 /dev/ttyACM0 -cmd "hf search"	(c) web Query: "Scan http://test.site using wafw00f with maximum verbosity and do not follow HTTP redirects." Tool_name: wafw00f Optional_args: ["-noredirect -v": null, "-verbose -v+": null] Positional_args: ["http://test.site"] GT_cmd: wafw00f --noredirect --verbose http://test.site	(c) hardware Query: "Use qemu-system-arm to emulate a system with 2 CPU threads, 1024 MB RAM, and load the device tree blob 'board.dtb'. Tool_name: qemu-system-arm Optional_args: ["-smp": "2", "-m": "1024", "-dtb": "board.dtb"] Positional_args: [] GT_cmd: qemu-system-arm -smp 2 -m 1024 -dtb board.dtb	(d) reverse-engineering Query: "Disassemble the dex file app.dex into small files in the directory 'output_small', forcing overwrite if it already exists." Tool_name: d2baksmali Optional_args: ["-force -f": null, "-output -o": "output_small"] Positional_args: ["app.dex"] GT_cmd: d2baksmali --force -output output_small app.dex
(d) bluetooth Query: "Start scanning for nearby Bluetooth devices and display their addresses and names." Tool_name: bluetoothctl Optional_args: [] Positional_args: ["scan", "on"] GT_cmd: bluetoothctl scan on	(d) database Query: "Use sqlmap to enumerate all databases available on the backend database of the vulnerable web application at http://test.site/item.php?id=1." Tool_name: sqlmap Optional_args: ["-u": "http://test.site/item.php?id=1", "-db": null] Positional_args: [] GT_cmd: sqlmap -u "http://test.site/item.php?id=1" --db	(d) wireless Query: "Use sakis3g to initiate an interactive connection session." Tool_name: sakis3g Optional_args: ["-interactive": null] Positional_args: ["connect"] GT_cmd: sakis3g --interactive connect	(b) forensics Query: "Convert 'malware.dex' to Java without decoding any resources and suppress all source decompilation, saving only the resource-less structure." Tool_name: jadx Optional_args: ["-no-res -r": null, "-no-src -s": null] Positional_args: ["malware.dex"] GT_cmd: jadx --no-res --no-src malware.dex
(e) sdr Query: "Use rtl-sdr-scanner to scan from 100 MHz to 200 MHz with 5 sweeps, saving results to output.rf.csv." Tool_name: rtl-sdr-scanner Optional_args: ["-end -e": "200", "-start -s": "100", "-sweeps -s": "5"] Positional_args: ["output.rf.csv"] GT_cmd: rtl-sdr-scanner --end 200 --start 100 --sweeps 5 output.rf.csv	(e) voip Query: "Run rtpnrtmsound with the -v flag on the stapler.wav file found in /usr/share/rtpnrtmsound/." Tool_name: rtpnrtmsound Optional_args: ["-verbose -v": null] Positional_args: ["/usr/share/rtpnrtmsound/stapler.wav"] GT_cmd: rtpnrtmsound --verbose /usr/share/rtpnrtmsound/stapler.wav	(a) post-exploitation Query: "Display the privileges of user 'eve' as they would apply to the command 'cat /etc/shadow', using non-interactive mode." Tool_name: sudo Optional_args: ["-list -l": null, "-non-interactive -n": null, "-other-user -U": "eve"] Positional_args: ["cat", "/etc/shadow"] GT_cmd: sudo --list --non-interactive -U eve cat /etc/shadow	(c) reporting Query: "Record my desktop with a delay of 5 seconds before starting, output the video as 'demo.ogv', and disable sound recording." Tool_name: recordmydesktop Optional_args: ["-delay": "5", "-no-sound": null, "-output -o": "demo.ogv"] Positional_args: [] GT_cmd: recordmydesktop --delay 5 --no-sound --output demo.ogv
(f) social-engineering Query: "Create an access point on channel 6 with ESSID 'TotallyNotATrap', enable the Caffe Latte WEP attack, set the WEP flag in beacons to 1, and use interface wlan0mon." Tool_name: aircrack-ng Optional_args: ["-channel -c": "6", "--essid -e": "TotallyNotATrap", "--cafe-latte", null, "-wep-flag": "1"] Positional_args: ["wlan0mon"] GT_cmd: aircrack-ng --channel 6 --ssid TotallyNotATrap --cafe-latte --wep-flag 1 wlan0mon		(b) passwords Query: "Run ncrack with maximum speed template (-TS), targeting SSH on 10.0.0.5, using usernames from users.txt and passwords from pass.txt, and stop after first successful login." Tool_name: ncrack Optional_args: ["-f": null, "-T": "5", "-P": "pass.txt", "-U": "users.txt"] Positional_args: ["ssh/10.0.0.5"] GT_cmd: ncrack -f TS -P pass.txt -U users.txt ssh/10.0.0.5	(d) gpu Query: "Crack SHA1 password hashes from hashes.txt using GPU device 0 and the wordlist wordlist.txt with oclgpuaccel." Tool_name: oclgpuaccel Optional_args: ["-d": "0", "-p": "c", "-sha1"] Positional_args: ["hashes.txt", "wordlist.txt"] GT_cmd: oclgpuaccel -d 0 -t sha1 hashes.txt wordlist.txt

Figure 6: **Overview of KaliBench samples.** Representative examples spanning all five security phases and 23 evaluation dimensions are shown. Zoom in for finer details.

6. Do **not** include explanations, commentary, or extraneous text.
7. Missing flags, incorrect option-value pairs, or omitted positional arguments are penalized.

Output Format
<pre><output> [command] </output></pre>

D KaliBench Samples in Detail

This section provides a detailed overview of KaliBench. As illustrated in Figure 6, the benchmark is structured around five core security phases, covering 23 fine-grained evaluation dimensions. Each instance consists of a natural language query, its corresponding tool invocation (including optional and positional arguments), and a ground-truth command. This design enables systematic evaluation of a model’s ability to interpret, decompose, and execute security-oriented tasks. Overall, KaliBench captures a diverse range of real-world security workflows.

E Terminal verification Details

We perform sample-wise terminal verification by executing each `ground_truth_command` in a controlled Docker sandbox (with Kali Linux environment). For each command, we create an isolated working directory to avoid side effects, and run it via `subprocess` with a fixed timeout and resource limits. During execution, we capture `stdout`, `stderr`, `exit_code`, and runtime signals.

After execution, we assign labels based on simple rules:

- **PASS_EXECUTED**: exit code = 0
- **TIMEOUT**: command exceeds time limit
- **REJECT_HALLUCINATED_TOOL**: tool not found (e.g., exit code 127 or error patterns)
- **REJECT_HALLUCINATED_PARAMETER**: invalid or unsupported options detected from error messages
- **PASS_RUNTIME_ERROR**: non-zero exit code without clear hallucination signals

We use regex-based pattern matching on stderr/stdout to detect tool and parameter errors, ensuring that obvious hallucinations can be filtered out even if the command executes.

A brief Pseudo-code can further show our execution process:

```
# Input: JSONL dataset of query-command pairs
# Output: execution logs with classification labels
for each row in dataset:
    # 1. Prepare command
    cmd = row["ground_truth_command"]

    # 2. Create isolated sandbox
    workdir = create_sandbox(cmd)

    # 3. Execute command with timeout
    result = run(cmd, timeout=20s, in workdir)
    stdout, stderr = result.stdout, result.stderr
    exit_code = result.exit_code

    # 4. Classify execution outcome
    if result.timed_out:
        label = TIMEOUT

    elif exit_code == 127 or match(tool_error_pattern, stderr, stdout):
        label = REJECT_HALLUCINATED_TOOL

    elif match(parameter_error_pattern, stderr, stdout):
        label = REJECT_HALLUCINATED_PARAMETER

    elif exit_code == 0:
        label = PASS_EXECUTED

    else:
        label = PASS_RUNTIME_ERROR

    # 5. Save execution log
    save {
        command, stdout, stderr,
        exit_code, label, workdir
    }
```

This process filters out commands with hallucinated tools or invalid parameters, while retaining executable commands and partially valid ones that are **syntactically correct but may fail at runtime due to placeholder inputs** (e.g., using non-existent targets such as `curl https://example.com`).

F Representative Invalid Cases Identified During Manual Review

To further substantiate the error types summarized in the main text, we provide representative invalid cases identified during manual review.

(a) Malformed CLI options. For the query “Launch debug_me via qemu-aarch64_be and wait for a GDB connection on port 1234,” the correct command is “qemu-aarch64_be -g 1234 debug_me”, whereas the annotated ground truth uses the malformed option --g instead of -g.

(b) Omission of query-required arguments. For the query “Generate shell completion script for kustomize so it can be used with bash,” the correct command is “kustomize completion bash”, but the annotated ground truth uses only “kustomize completion” and omits the shell argument, which is specifically asked by the query.

(c) Semantic mismatches in argument use. We observe three common subtypes. First, the annotated command may use a flag with the wrong function: for the query “Run Git with the working directory set to /home/user/project,” the correct command is “git -C /home/user/project status”, while the ground truth uses work-tree, which changes the repository work tree rather than the working directory. Second, the annotated command may reverse operand order: for the query “converting mac.txt to unix3.txt”, the correct form is dos2unix -n mac.txt unix3.txt, while the ground truth swaps input and output. Third, the annotated command may use the wrong argument type: for the query “Execute svn’s sync subcommand on the local mirror located at /srv/svn/replica”, the correct command is “svn sync file:///srv/svn/replica”, while the ground truth supplies the plain path “/srv/svn/replica” where a URL is required.

G Experiment Details

G.1 Evaluated Model Details.

We evaluated open-source models with diverse sizes and architectures from 8B to 753B parameters. Models were either run locally using weights obtained from their official Hugging Face repositories or accessed via APIs. Table 4 summarizes the evaluated models, their parameter counts, inference modes, and Hugging Face repositories.

We used a fixed temperature of 0.2 and a maximum context length of 16,384 tokens, reserving 8,192 tokens for the input and 8,192 tokens for the output. When the combined prompt and tool documentation exceeded the reserved input budget, the tool documentation was truncated to fit within the input limit. Models with fewer than 72B parameters were evaluated on our 4× A100 GPU server using the vLLM inference engine with BF16 precision. Models with 72B or more parameters were evaluated through the OpenRouter API.

G.2 Training Setup.

We fine-tuned RedSage-Qwen3-8B-Ins using parameter-efficient LoRA adapters on a single NVIDIA H200 GPU. For both SFT and GRPO training, we used LoRA rank 64, LoRA alpha 128, and zero LoRA dropout, applying adapters to the attention and MLP projection layers: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj. Training used bfloat16 precision, gradient checkpointing, and the 8-bit AdamW optimizer with learning rate 5×10^{-6} , weight decay 10^{-3} , Adam betas (0.9, 0.999), gradient clipping at 1.0, and a linear learning-rate schedule with 10% warmup. We trained for 2 epochs. The software stack included Transformers 4.57.6, Unsloth 2026.4.6, PEFT 0.19.1, and Python 3.12.13.

For the SFT model, we trained from the RedSage-Qwen3-8B-Ins checkpoint with a per-device batch size of 4 and gradient accumulation of 8, giving an effective batch size of 32. The maximum sequence length was 16,384 tokens, with packing disabled and padding-free training enabled. The SFT objective used the standard negative log-likelihood loss. SFT training required ~32 minutes.

For the GRPO-trained model, we used the same optimizer, LoRA, precision, and batch settings as SFT. GRPO used 8 generations per prompt, a generation batch size of 32, and group-based reward scaling. The maximum prompt length was 5,549 tokens, and the maximum completion length was 6,739 tokens. We used the BNPO-style GRPO loss with clipping parameter $\epsilon = 0.2$ and KL coefficient $\beta = 0.001$. Generation during GRPO was accelerated with colocated vLLM, using tensor parallel size 1 and GPU memory utilization 0.3. GRPO training required ~28 hours when initialized directly from RedSage-Qwen3-8B-Ins, and ~17 hours when initialized from our SFT checkpoint.

Model Name	Size	Inference	Hugging Face Repository
Llama3.1-Instruct	8B	local vLLM	meta-llama/Llama-3.1-8B-Instruct
Foundation-Sec-Instruct	8B	local vLLM	fdtn-ai/Foundation-Sec-8B
Foundation-Sec-Reasoning	8B	local vLLM	fdtn-ai/Foundation-Sec-8B-Reasoning
Llama-Primus-Base	8B	local vLLM	trend-cybertron/Llama-Primus-Base
Llama-Primus-Merged	8B	local vLLM	trendmicro-ailab/Llama-Primus-Merged
RedSage-Qwen3-8B-Ins	8B	local vLLM	RISys-Lab/RedSage-Qwen3-8B-Ins
RedSage-Qwen3-8B-DPO	8B	local vLLM	RISys-Lab/RedSage-Qwen3-8B-DPO
RedSage-K (SFT)	8B	local vLLM	RISys-Lab/RedSage-K-SFT
RedSage-K (GRPO)	8B	local vLLM	RISys-Lab/RedSage-K-GRPO
RedSage-K (SFT+GRPO)	8B	local vLLM	RISys-Lab/RedSage-K-SFT-GRPO
Qwen-3-8B	8B	local vLLM	Qwen/Qwen3-8B
GPT-OSS-20B	21B	local vLLM	openai/gpt-oss-20b
Mistral-3.2-24B	24B	local vLLM	mistralai/Mistral-Small-3.2-24B-Instruct-2506
Gemma-3-27B	27B	local vLLM	google/gemma-3-27b-it
Qwen-3-32B	32B	local vLLM	Qwen/Qwen3-32B
LLaMA-3.3-Instruct	70B	local vLLM	meta-llama/Llama-3.3-70B-Instruct
Llama-Primus-Nemotron-70B-Instruct	71B	local vLLM	trend-cybertron/Llama-Primus-Nemotron-70B-Instruct
Qwen-2.5-Instruct	72B	OpenRouter API	Qwen/Qwen2.5-72B-Instruct
Qwen3-Coder-Next	80B	OpenRouter API	Qwen/Qwen3-Coder-Next
GPT-OSS-120B	120B	OpenRouter API	openai/gpt-oss-120b
DeepSeek-V3.2	685B	OpenRouter API	deepseek-ai/DeepSeek-V3.2
GLM-5.2	753B	OpenRouter API	zai-org/GLM-5.2

Table 4: Evaluated open-weight model configurations, parameter counts, inference platforms, and model sources. The inference platform used for each configuration is specified in the table.

H Model Performance Difference across Execution Modes

Table 5 provides a model-wise decomposition of performance changes across execution modes, complementing the raw-score analysis in Table 1.

A consistent pattern is that improvements in tool selection do not propagate proportionally to exact command correctness. Although Tool Δ_1 is large across all models (23.2 pp on average), Exact Δ_1 remains much smaller (5.9 pp on average). This indicates that even if the tool name can be settled, the remaining errors are still dominated by argument selection, value binding, and command formatting. Thus, restricted mode mainly reduces tool-choice ambiguity, but does not substantially solve command construction.

The restricted-to-hinted transition shows the opposite behavior. Tool Δ_2 is small, unstable, and often negative: 11 out of 24 models show degraded Tool Accuracy after adding full tool manuscripts, with *Gemma-3-it-27B* dropping by 10.3 pp. In contrast, argument-related metrics improve substantially, with Optional F1 increasing by about 39.1 pp and Positional F1 by about 21.9 pp on average. This asymmetry suggests that tool manuscripts are not primarily useful for identifying tools; instead, they help models resolve optional-argument ambiguity, flag selection, value specification, and positional ordering.

Exact Correct further confirms this mechanism. Exact Δ_2 averages 44.8 pp, nearly eight times larger than Exact Δ_1 , showing that argument correction is far more effective than tool disambiguation for producing fully correct commands. Model-wise, the largest Exact gains are more closely associated with large Optional Δ_2 than with large Tool Δ_1 , suggesting that optional-argument modeling is the key factor determining whether partial command understanding becomes exact command reconstruction.

The *RedSage-K* variants show a distinct delta pattern: despite strong raw performance in the *U* and *R* settings, they benefit less from mode switching than many baselines. In particular, *RedSage-K (GRPO)* and *RedSage-K (SFT+GRPO)* exhibit smaller Optional, Positional, Total, and Exact Δ s than strong general-purpose models. This suggests that KaliBench post-training improves intrinsic

Table 5: **Performance deltas across execution modes.** We report differences between three execution modes: Unrestricted (U), Restricted (R), and Hinted (H). Δ_1 denotes (R−U), Δ_2 denotes (H−R), and Δ_3 denotes (H−U), all measured in percentage points (pp). $\dagger$ indicates thinking-enabled models, and $*$ denotes cybersecurity-tuned models. **Bold/underline** mark the largest and second-largest deltas within each size group. Model order follows Table 1.

Model	Size	Tool score Δ (pp)			Optional F1 Δ (pp)			Positional F1 Δ (pp)			Total score Δ (pp)			Exact Δ (pp)		
		Δ_1	Δ_2	Δ_3	Δ_1	Δ_2	Δ_3	Δ_1	Δ_2	Δ_3	Δ_1	Δ_2	Δ_3	Δ_1	Δ_2	Δ_3
Compact Models ($\leq 8B$)																
Foundation-Sec-Ins*	8	26.00	-5.70	20.30	3.90	38.20	42.10	7.00	21.30	28.30	12.30	17.90	30.20	4.40	37.10	41.50
Llama3.1-Ins	8	32.00	1.70	33.70	4.80	45.50	50.30	8.40	23.90	32.30	15.10	23.70	38.80	5.70	44.50	50.20
Llama-Primus-Base*	8	23.40	3.40	26.80	2.60	41.60	44.20	5.60	22.10	27.70	10.50	22.40	32.90	3.50	41.00	44.50
Foundation-Sec-Rsn [†]	8	25.80	<u>3.10</u>	28.90	4.30	46.40	50.70	4.90	28.30	<u>33.20</u>	11.70	25.90	<u>37.60</u>	5.30	48.00	53.30
Llama-Primus-Merged*	8	27.20	1.80	<u>29.00</u>	4.10	43.30	47.40	6.60	23.90	30.50	12.60	23.00	35.60	4.70	44.20	48.90
RedSage-DPO*	8	<u>28.30</u>	0.60	28.90	6.00	39.30	45.30	9.20	22.60	31.80	<u>14.50</u>	20.90	35.40	7.20	42.70	49.90
Qwen3	8	26.70	0.70	27.40	4.10	44.80	48.90	7.10	24.90	32.00	12.70	23.40	36.10	5.80	<u>49.00</u>	<u>54.80</u>
Qwen3 [†]	8	25.40	0.50	25.90	4.30	45.20	49.50	6.60	<u>26.90</u>	33.50	12.10	<u>24.20</u>	36.30	<u>6.80</u>	53.20	60.00
RedSage-Ins*	8	26.70	0.70	27.40	4.50	38.10	42.60	7.60	21.90	29.50	12.90	20.20	33.10	6.40	41.10	47.50
RedSage-K (GRPO)* [†]	8	16.80	-0.20	16.60	2.00	31.10	33.10	3.10	11.40	14.50	7.20	14.20	21.40	4.90	33.70	38.60
RedSage-K (SFT)*	8	17.80	1.20	19.00	2.40	35.30	37.70	1.50	23.60	25.10	7.30	20.00	27.30	3.80	42.70	46.50
RedSage-K (SFT+GRPO)* [†]	8	17.10	-2.50	14.60	1.80	28.60	30.40	4.00	8.20	12.20	7.60	11.40	19.00	5.20	32.00	37.20
Mid-size Models (20-32B)																
Gemma-3-it	27	23.90	-10.30	13.60	3.30	38.90	42.20	6.10	22.10	28.20	11.10	17.00	28.10	5.10	46.50	51.60
GPT-OSS [†]	20/3.6	22.80	0.80	<u>23.60</u>	3.40	45.70	49.10	6.50	28.40	34.90	10.90	24.90	35.80	5.00	53.50	<u>58.50</u>
Mistral-Small-3.2	24	25.00	<u>0.60</u>	25.60	3.90	43.30	47.20	<u>6.60</u>	<u>24.00</u>	<u>30.60</u>	11.80	<u>22.70</u>	<u>34.50</u>	6.70	<u>52.50</u>	59.20
Qwen3	32	22.80	-0.80	22.00	5.50	42.30	<u>47.80</u>	7.10	23.50	<u>30.60</u>	11.80	21.70	33.50	7.10	52.10	59.20
Qwen3 [†]	32	<u>24.40</u>	-1.60	22.80	<u>4.40</u>	40.80	45.20	6.10	22.60	28.70	<u>11.70</u>	20.60	32.30	<u>6.90</u>	50.90	57.80
Large-scale Models ($\geq 70B$)																
Llama-Primus-Ins*	70	<u>23.60</u>	-4.30	19.30	4.40	41.80	46.20	<u>7.40</u>	23.20	30.60	11.80	20.20	32.00	6.20	46.20	52.40
Llama-3.3-Ins	70	24.40	-2.40	22.00	4.20	41.80	46.00	6.20	24.40	30.60	<u>11.60</u>	21.20	32.80	6.10	49.00	55.10
GPT-OSS [†]	120/5.1	22.00	<u>0.40</u>	22.40	4.90	39.30	44.20	7.30	<u>24.00</u>	<u>31.30</u>	11.40	21.20	<u>32.60</u>	<u>8.00</u>	48.90	56.90
Qwen3-Coder-Next [†]	80/3	22.70	-0.40	<u>22.30</u>	3.10	39.50	42.60	6.80	22.50	29.30	10.80	<u>20.50</u>	31.30	4.80	48.90	53.70
Qwen2.5-Ins	72	21.40	-0.60	20.80	3.40	40.40	43.80	5.20	21.10	26.30	10.00	20.20	30.20	4.80	50.40	<u>55.20</u>
DeepSeek-V3.2 [†]	685/37	22.50	-1.20	21.30	5.00	31.50	36.50	6.60	18.70	25.30	11.40	16.30	27.70	7.20	44.30	51.50
GLM-5.2 [†]	753	7.90	0.70	8.60	-3.40	16.50	13.10	27.50	11.80	39.30	10.70	9.60	20.30	10.70	22.80	33.50
Metric Average	–	23.19	-0.58	22.62	3.62	39.13	42.75	7.13	21.89	29.01	11.31	20.14	31.45	5.93	44.80	50.73

Table 6: **Effect of RLVR on RedSage-K across evaluation modes.** Scores are Total Score percentages. Δ denotes the change from the original *RedSage-Ins* model in percentage points.

Mode	RedSage-Ins	RedSage-K (GRPO)	Δ_{GRPO}	RedSage-K (SFT+GRPO)	$\Delta_{SFT+GRPO}$
Unrestricted	56.36	67.40	+11.04	70.31	+13.95
Restricted	69.31	74.65	+5.34	77.93	+8.62
Hinted	89.53	88.77	-0.76	89.34	-0.19

command understanding, but the *RedSage-K* variants may be less effective at exploiting full tool documentation, consistent with their weaker *Hinted*-mode gains in the main results.

Overall, the delta analysis shows that NL-to-CLI errors are not reduced uniformly across components. The main model-wise differentiator is not the magnitude of tool-selection gains, but how efficiently models use documentation to repair argument errors and produce exact commands.

I RLVR Failure Analysis

Our paired analysis of all 5,000 evaluation examples across the three settings identifies three recurring RLVR failure modes: invalid optional arguments or option–value pairs, incorrect positional-argument construction, and regressions on examples already solved by the baseline. The model often selects the correct tool but generates incorrect argument forms, such as `msfvenom -list-formats` instead of `msfvenom -list formats`.

Although the GRPO and SFT+GRPO variants of *RedSage-K* improve Total Score in U and R, they reduce it in H by 0.76 and 0.19 percentage points, respectively, relative to the original *RedSage-Ins* model. In H, the GRPO variant corrects 537 baseline exact-match failures but regresses on 562 baseline successes. The degradation is concentrated in commands requiring optional arguments or a mixture of optional and positional arguments.

SFT initialization mitigates these regressions. Compared with GRPO alone, SFT+GRPO improves Total Score by 2.91 points in U and 3.28 points in R, while reducing H-mode exact-match regressions from 562 to 488.



Across security dimensions, both RLVR variants show broad improvements in U and R. We find no robust evidence of a dimension in which RLVR systematically fails to improve; the small variations in H do not form a consistent dimension-specific pattern.

To assess whether KaliBench captures meaningful CLI behaviors rather than being dominated by generic flags, we analyze option frequencies across evaluation, train, and full sets. In all, there are 3,074 different optional arguments in the evaluation set, 2,739 in the train set, and 4,378 in the full set. Figure 7 shows the top 40 options in the evaluation subset. As expected, broadly supported meta-flags such as `--verbose|-v` and `--help|-h` appear most frequently. However, the distribution also covers security-relevant CLI patterns, including network and web parameters (`--port|-p`, `--domain|-d`, `--url|-u`), interface/address controls (`--interface|-i`, `--dst-addr|-d`), and data/output handling flags (`--file|-f`, `--format|-f`, `--output|-o`, `--reportformat`).

32

Table 7: Best- and worst-representative tools aggregated from the scoring files of all models.

title	avg_total	avg_tool	avg_optional	avg_positional	avg_exact	count	score_sum
<i>Best Reps</i>							
d2j-std-apk	1.000	1.000	1.000	1.000	1.000	72	5.000
rfdump	1.000	1.000	1.000	1.000	1.000	72	5.000
truecrypt2john	1.000	1.000	1.000	1.000	1.000	72	5.000
jsql	0.998	1.000	0.995	1.000	0.995	216	4.989
reconspider	0.998	1.000	1.000	0.993	0.993	288	4.984
<i>Worst Reps</i>							
steghide	0.560	1.000	0.347	0.333	0.319	72	2.560
above	0.490	0.747	0.357	0.368	0.319	288	2.281
exiflooter	0.491	0.719	0.379	0.375	0.191	288	2.155
hashrat	0.464	0.524	0.273	0.593	0.184	288	2.038
nikto	0.347	0.986	0.012	0.042	0.000	72	1.386

Notes: *count* denotes the number of times that a tool appears in the files. *score_sum* is the unweighted sum of all averaged metrics.

workflows. These security-specific flags demonstrate that KaliBench covers diverse and operationally meaningful command patterns, making it suitable for evaluating fine-grained CLI generation in realistic cybersecurity settings.

K Complete List of Best-Worst Performed Tools

Table 7 reports representative high- and low-performing tools after aggregating scores across 24 open-weight model configurations and three evaluation modes. For each tool, scores are averaged over all associated evaluation records.

High-performing tools show consistently strong performance across all sub-metrics. For example, d2j-std-apk, rfdump, and truecrypt2john achieve perfect average scores, while jsql and reconspider remain near-perfect with avg_exact scores of 0.995 and 0.993, respectively. Their high avg_optional and avg_positional scores indicate that the evaluated models reliably construct arguments for these tools.

By contrast, low-performing tools reveal substantial weaknesses in argument generation. Although their avg_total scores range from 0.347 to 0.560, their Exact scores remain much lower. The clearest example is nikto, which has near-perfect tool identification (avg_tool = 0.986) but zero exact correctness (avg_exact = 0.000), alongside very low avg_optional and avg_positional scores (0.012 and 0.042). Similar gaps appear for steghide, above, and exiflooter, showing that strong performance on an individual component is insufficient for complete command correctness.

L Details about Metrics Calculation

L.1 Command-level Scoring for Evaluating Models

For each example, we extract the predicted command from the <output> tag and parse both the ground-truth and predicted commands into: <tool name>, <optional flags>, and <positional arguments> to know which flags take values and which flags are aliases.

Tool score Tool_score is a binary indicator: it is 1 if the predicted tool name exactly matches the ground-truth tool name, and 0 otherwise.

Optional-argument F1. Ground-truth optional flags are grouped into alias sets (e.g., ‘-h’ | ‘-help’). A predicted flag is counted as correct for a group if it matches any alias in that group. Let G be the number of ground-truth groups, N_{pred} be the total number of predicted optional flags, and H be the number of ground-truth groups that are “hit” by at least one predicted alias. We define

$\text{precision}_{\text{opt}} = H/N_{\text{pred}}$ and $\text{recall}_{\text{opt}} = H/G$, and compute

$$\text{optional_f1} = \frac{2 \times \text{precision}_{\text{opt}} \times \text{recall}_{\text{opt}}}{\text{precision}_{\text{opt}} + \text{recall}_{\text{opt}}}.$$

If both ground truth and prediction have no optional flags, we set *optional_f1* to 1; if ground truth command has no optional flags while the predicted command has, we set it to 0.

Positional-argument F1. The ground-truth and predicted positional arguments form 2 sets. We match each predicted argument to an identical ground-truth argument at most once: every successful match counts as a true positive *TP*; any remaining unmatched predicted arguments are false positives *FP*; and any remaining unmatched ground-truth arguments are false negatives *FN*. We then define $\text{precision}_{\text{pos}} = TP/(TP + FP)$ and $\text{recall}_{\text{pos}} = TP/(TP + FN)$, and compute

$$\text{positional_f1} = \frac{2 \text{precision}_{\text{pos}} \text{recall}_{\text{pos}}}{\text{precision}_{\text{pos}} + \text{recall}_{\text{pos}}}.$$

If both lists are empty, we set *positional_f1* to 1; if exactly one is empty, we set it to 0.

Total score and Exact match Total score is the average of tool score, optional f1 and positional f1. And Exact match evaluates end-to-end command correctness which is 1 for generating a correct canonicalized command, and 0 for any error in parameters.

L.2 Dimensional Score Calculation

To analyze model performance across functional security domains, we leverage Kali’s metapackage taxonomy (Section 3) to assign each tool to one or more of $D = 23$ capability dimensions. Each query–command pair is therefore annotated with a multi-hot vector $\mathbf{v}^{(i)} \in \{0, 1\}^D$ indicating the dimensions to which its ground-truth tool belongs.

For each evaluation instance i , we project its scalar prediction score $s^{(i)} \in [0, 1]$ onto the corresponding capability dimensions by element-wise multiplication:

$$\mathbf{s}^{(i)} = s^{(i)} \cdot \mathbf{v}^{(i)}.$$

This assigns the score $s^{(i)}$ to all dimensions associated with the tool, while all other dimensions receive zero. Summing $\mathbf{s}^{(i)}$ over all instances yields per-dimension performance scores for each model, enabling fine-grained analysis across the 23 cybersecurity capability dimensions.

Example. Consider a tool that is associated with three capability dimensions in Kali’s taxonomy, corresponding to dimensions 2, 4, and 6 out of the total $D = 23$. Its multi-hot encoding vector is therefore

$$\mathbf{v}^{(i)} = (0, 1, 0, 1, 0, 1, 0, \dots, 0) \in \{0, 1\}^{23},$$

where the nonzero entries indicate the relevant capability dimensions and all remaining positions are zero. If the model assigns this instance a scalar prediction score $s^{(i)} = 0.8$, the projected dimensional score vector becomes

$$\mathbf{s}^{(i)} = 0.8 \cdot \mathbf{v}^{(i)} = (0, 0.8, 0, 0.8, 0, 0.8, 0, \dots, 0).$$

Thus, the same prediction score is assigned to each of the associated capability dimensions (2, 4, and 6), while having no effect on the others. Aggregating such vectors across all evaluation instances yields cumulative performance scores for each capability dimension.

M Detailed Results for Proprietary Models and Agentic Scaffold

Table 8 reveals a different component-score pattern from the aggregate open-weight results: both standalone proprietary models achieve lower Pos-F1 than Opt-F1 despite high tool-selection accuracy. This pattern also appears in the Codex configuration, suggesting that argument construction remains challenging across both standalone and scaffolded settings. Moreover, Exact Correct (EC) reflects the joint correctness of the entire command construction process, requiring correct tool selection, flag–value bindings, and positional arguments. Errors across these components accumulate, yielding EC scores below the lowest component score for all three modes. These findings underscore the value of fine-grained evaluation in explaining the gap between individual component performance and complete command correctness.

Table 8: **Complete results for proprietary models and agentic scaffolds in the Unrestricted setting.** We used the following abbreviations: Tool Accuracy (TA), Optional-Argument F1 (Opt-F1), Positional-Argument F1 (Pos-F1), Total Score (TS), and Exact Correct (EC). All metric scores are percentages. Bold shows the best.

Model / Scaffold	Answered queries	Blocked / Missing	TA	Opt-F1	Pos-F1	TS	EC Answered	EC All 5K
GPT-5.6-Sol	4,988 (99.76%)	12 (0.24%)	92.54	87.42	71.51	83.83	61.83	61.68
Claude Opus 5	3,675 (73.50%)	1,325 (26.50%)	90.67	85.95	68.57	81.73	59.89	44.02
Codex (GPT-5.5, xhigh)	4,633 (92.66%)	367 (7.34%)	91.28	82.39	63.46	79.04	55.77	51.68

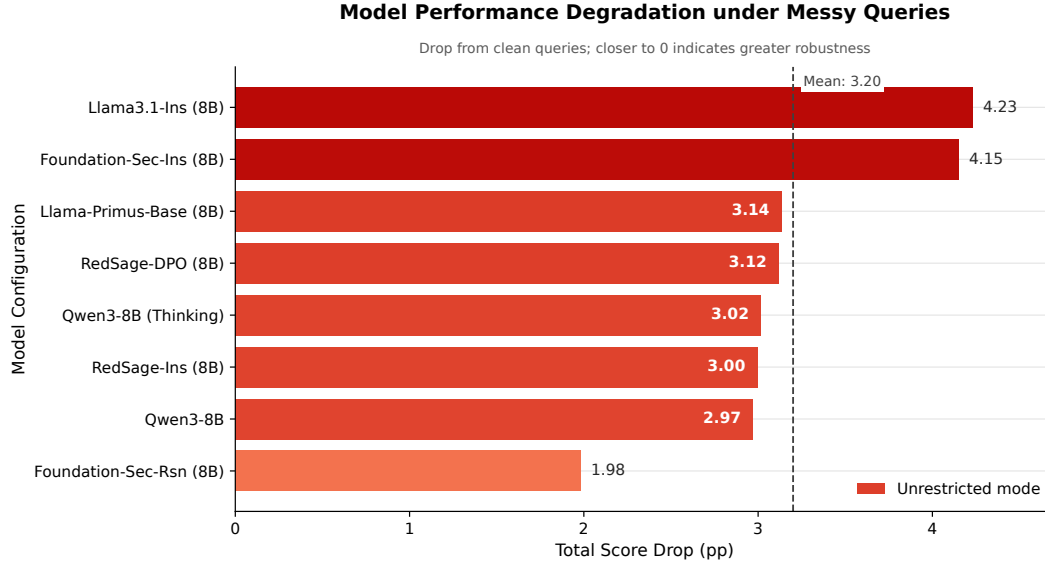


Figure 8: Model-wise Sensitivity to Messy Queries

N Model-wise Sensitivity to Messy Queries

We examine whether sensitivity to messy user queries is shared across the evaluated model configurations. Under the Unrestricted setting, we compare each configuration’s Total Score on Messy Query inputs with its own performance on the corresponding Clean Query inputs.

Fig 8 shows that every tested configuration experiences a decrease in Total Score. Most configurations lose approximately 3 percentage points, while the two most sensitive configurations lose more than 4 percentage points. Even the smallest observed decrease is 1.98 percentage points. This consistent trend across the tested configurations suggests that degradation to messy queries is not confined to a single model family, and how to deal with messy user queries remain a systematic challenge for natural-language-to-CLI translation task.

O Limitations

KaliBench is designed as a deterministic and reproducible benchmark for evaluating NL-to-CLI function calling, and it inherits several limitations intrinsic to the CLI ecosystem and its documentation sources. Ground-truth commands are derived from Kali Linux tool manuals available at dataset construction time. As these manuals evolve across versions through the addition, deprecation, renaming, or reinterpretation of flags, some canonicalized commands may become outdated as upstream documentation changes. In practice, such effects are limited, as tools often maintain backward compatibility, and evaluation in the hinted setting is further insulated from drift by explicitly providing models with the tool documentation to follow.

In addition, real-world tool documentation may occasionally contain inconsistencies or errors. For example, the `regtree` manual lists the `-V` option with conflicting semantics (`-V`, `--no-values` and

-V, --version), and several tools in the *Impacket* family document atypical short-option forms (e.g., -aesKey) that differ from common usage conventions (e.g., --aesKey). Such cases reflect the diversity and historical evolution of CLI interfaces rather than issues specific to KaliBench, and they make fully deterministic canonicalization inherently approximate when documentation diverges from observed behavior or when multiple variants coexist.

Our LLM-as-Verifier pipeline substantially filters hallucinated flags and arguments by grounding validation in official manuals, but it remains imperfect; human verification of a random subset indicates that residual errors are limited. Similarly, the alias-aware scoring mechanism relies on semi-automatically extracted alias groups and cannot capture all historical, distribution-specific, or community-adopted variants.

Taken together, these considerations position KaliBench as a controlled and well-defined abstraction of real-world CLI tool invocation. It is designed to support systematic, fine-grained evaluation of NL-to-CLI translation, while complementing execution-based or agentic evaluations in operational cybersecurity environments.